

claude-windows / 78dabe12-41fd-46c2-ae90-ab74bdaec3c6

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\78dabe12-41fd-46c2-ae90-ab74bdaec3c6\subagents\workflows\wf_18dcd7ff-5ae\agent-a689255126261a812.jsonl`  
- SHA-256: `444e1c48151adc0a9f8ae6e2f8cd53fb547222ed1a7ab1aa643c32ab7ce42c56`  
- Source modified: `2026-06-22T10:28:04+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_18dcd7ff-5ae`  
- session_id: `78dabe12-41fd-46c2-ae90-ab74bdaec3c6`
```

Transcript

1. user (2026-06-22T10:25:19.418Z)

CONTEXT ? a GCS corpus-migration work item just completed on branch work/golden-corpus-hygiene (PR #316, repo Khelsutra/badminton-highlight-indexer). Goal: run ALL training+inference from the cloud with ALL data in gs://khelsutra-rally-corpus so the local Windows box + external drives stop being blockers.

AUTHORITATIVE DOC TO REVIEW: docs/DATA_IN_GCS.md (the corpus source-of-truth map).
RELEVANT CODE: backend/worker/__main__.py (cmd_infer ~157, cmd_train ~332, --corpus-uri ~473), backend/storage/ref.py (URI schemes/resolve_input_uri), deploy/gcp/README.md, docs/SETUP_NEW_MACHINE.md, docs/archives/past_projects/DECOUPLED_COMPUTE/.

VERIFIED BUCKET STATE (already confirmed deterministically this session ? treat as ground truth, do NOT

re-run gcloud, it may not be authed for you):

- trajectories/ = 15 objects (<name>.csv)
- labels/ = 15 canonical objects only, named <YYYY-MM-DD>_<name>.rallies.csv (date = earliest mtime across C:/F: copies); the 7 old inconsistent (*_proxy / bare GX010128,mahadevpura-1) were MD5-verified byte-identical dupes and DELETED.
- manifests/golden.json = 15 entries, every trajectory+label gs:// ref RESOLVES (0 dangling).
- videos/ = mahadevpura_1.mp4 (523MB, demo of Drive->bucket) + 3 pre-existing (GX030094_proxy.mp4, mahadevpura-1.MP4, mahadevpura_smoke_180s.mp4). 14 of 15 golden videos NOT yet uploaded.
- worker train reads labels/(+videos/) and BUILDS the manifest on-box (does NOT read manifests/golden.json); manifests/golden.json is for EVAL. trajectories are a cache (training can regenerate via --trajectory-source detector).
- Open: Phase-2 videos (7 Drive proxies + 8 large F: full-MP4s, proxy-first), Phase-3 flip eval/litmus to GCS.
- Sibling threads (from memory): a golden-fixtures P3 thread needs these same golden traj+label CSVs; the served Gate-A litmus (#236, merged) is pinned to local paths; original-6 labels turned out recoverable from F:.

LENS = RISK / SAFETY AUDIT. What was risky or could bite us? Consider: the 7 deleted label objects (was deletion truly safe + is anything now unrecoverable?), the bracketed-path gcloud glob gotcha, the migration/copy scripts living in gitignored scratch/ (not reproducible by others / not in the PR), storage cost of ~70GB + egress, naming-convention edge cases (recycled GoPro names, date-collision), whether the cloud manifest format matches what any eval consumer expects. Rank by severity.

Return ONLY the structured verdict.

2. assistant (2026-06-22T10:25:22.783Z)

I'll audit this GCS corpus-migration work item through a risk/safety lens. Let me read the authoritative doc and relevant code.

3. user (2026-06-22T10:25:23.753Z)

```
1      # Data in GCS ? the corpus source-of-truth map
2
3      **Status:** LIVE / in-progress (migration not complete). **Owner-gated** decisions in
$7.
4      **Last inventoried:** 2026-06-22 against `gs://khelsutra-rally-corpus`.
5      **Goal (owner, 2026-06-22):** run **all** training + inference from the cloud with **all
data in GCS**,
6      so the local Windows box (and the intermittently-mounted external drive) is **no longer
a blocker**.
7
8      > **? Other sessions: this is where the data lives.** Pull from `gs://khelsutra-rally-
corpus`, not
9      > from anyone's `C:` / `F:` / Google Drive. If something you need isn't in the bucket
yet, it's in the
10     > **gap table ($5)** ? flag it / migrate it, don't hard-code a local path.
11
12     This doc is the **data map**. The *machine* runbooks are
[`SETUP_NEW_MACHINE.md`](SETUP_NEW_MACHINE.md)
13     (spin up a box) and [`deploy/gcp/README.md`](../deploy/gcp/README.md) (provision GPU +
train/infer).
14     The cloud engine itself was built by the (archived) DECOUPLED_COMPUTE project
15     (`docs/archives/past_projects/DECOUPLED_COMPUTE/`); this doc does **not** redesign it ?
it records
16     *where the bytes are* and *what's still missing*.
17
18     ---
19
20     ## 1. The bucket
21
22     | | |
23     |---|---|
24     | Bucket | **`gs://khelsutra-rally-corpus`** (single corpus bucket) |
25     | Project | `gen-lang-client-0306026826` (billing *Khelsutra Billing*) |
26     | Region | `asia-south1` (Mumbai), UBLA, co-located with the GPU VMs |
27     | Auth | SA `rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com`
(`roles/storage.objectAdmin`); key at `~/gcp/rally-worker-sa.json` (local only, **never
committed**). Interactive: `gcloud auth login`. |
28     | Size today | ~2.06 GB (mostly past dry-run reels/outputs ? **not** the full corpus) |
29
30     The engine is **backend-agnostic by URI** (`backend/storage/ref.py`): `gs://` (or
`gs://`),
31     `s3://`, `gdrive://`, or a bare local path. Pick the backend with **config + creds, no
code change**.
32
33     ---
34
35     ## 2. Canonical layout (the target contract)
36
37     The cloud worker's input/output contract (`backend/worker/__main__.py`; lineage:
38     `DECOUPLED_COMPUTE/04`):
39
40     | Prefix | Holds | Who reads/writes it |
41     |---|---|---|
42     | `videos/` | source videos / proxies (<clip>.mp4) | **input** to `worker infer` &
`worker train` |
43     | `labels/` | golden rally labels (<clip>.rallies.csv) | **input** ground truth to
```

```

`worker train` + all eval |
44 | `trajectories/` | cached WASB trajectory CSVs | **(PROPOSED ? see §7.2)** GPU-free
eval/litmus input |
45 | `weights/<version>/` | trained Gen-0 bundles (`weights.json` + a run `manifest.json`)
| **output** of `worker train` |
46 | `models/` | upstream pretrained weights (`wasb_badminton_best.pth.tar`) | input to the
WASB detector |
47 | `outputs/<video_id>/` | per-video inference outputs (intervals, etc.) | **output** of
`worker infer` |
48 | `highlights/`, `promo/` | rendered reels + promo artifacts | output / demo |
49
50 > **Note on the manifest.** `worker train` does **not** read a pre-staged corpus
`manifest.json` ? it
51 > *builds* one on-box at train time from `labels/` (+`videos/`), extracting trajectories
via the native
52 > detector (`--trajectory-source detector`). A `manifest.json` only appears as an
**output** inside each
53 > `weights/<version>/` bundle. The **eval/litmus** path is what wants a stable, path-
resolvable manifest
54 > (§7.3).
55
56 ---
57
58 ## 3. What's actually in the bucket today (2026-06-22)
59
60 > **Phase-1 landed 2026-06-22:** added `trajectories/` (15), 15 canonical
`labels/<date>_<name>.rallies.csv`,
61 > and `manifests/golden.json`. The table below is the **pre-Phase-1** snapshot + the
*remaining* gaps
62 > (videos, the 7 stale label objects to delete) ? see §7/§8.
63
64 | Prefix | Present | Drift / notes |
65 |---|---|---|
66 | `labels/` | 7 files: `GX010128`, `mahadevpura-1`, + `Badminton BXH 2_proxy`, `Boxhill
Doubles_proxy`, `GX020094_proxy`, `GX030094_proxy`, `mahadevpura-2_proxy` | ? **inconsistent
naming** (`_proxy` on some, not others); ? **missing** the original-6 labels + `GX010137`,
`GX010141` |
67 | `videos/` | 3: `GX030094_proxy.mp4`, `mahadevpura-1.MP4`, `mahadevpura_smoke_180s.mp4`
| ? corpus videos/proxies almost entirely absent |
68 | `trajectories/` | ? | ? **does not exist** (every trajectory is local-only) |
69 | `models/` | `wasb_badminton_best.pth.tar` | ? (? WASB-C3 weight-provenance still an
open legal item before any external share) |
70 | `weights/` | `0.1.0-l4/` | ? first real L4 train bundle |
71 | `outputs/` | one `/`, `parity/` | ? dry-run outputs |
72 | `highlights/`, `promo/` | reels + promo | ? |
73
74 **Drift summary:** the bucket has extra prefixes (`highlights/`, `promo/`, `models/`
alongside
75 `weights/`) and **inconsistent `labels/` naming**, and is **missing the bulk of the
corpus inputs**
76 (`videos/`, all `trajectories/`, most `labels/`). It reflects past *dry runs*, not a
complete corpus.
77
78 ---
79
80 ## 4. How the cloud consumes it
81
82 Provisioning + the verified commands live in
[`deploy/gcp/README.md`](../deploy/gcp/README.md) §3?§5.
83 The data-facing entry points (`backend/worker/__main__.py`):
84
85 ```bash
86 # config (no secrets in config; auth via GOOGLE_APPLICATION_CREDENTIALS):
87 # { "storage": { "backend": "local", "gcs": { "project": "gen-lang-client-0306026826"
} } }

```

```

88
89 # INFERENCE on one cloud video -> outputs/<video_id>/ in the bucket
90 python -m backend.worker infer \
91     --video-uri gcs://khelsutra-rally-corpus/videos/<clip>.mp4 \
92     --out-uri gcs://khelsutra-rally-corpus --segmenter wasb_hybrid --config config.json
93
94 # TRAIN from the bucket (reads labels/ [+ videos/], extracts trajectories, builds
manifest on-box,
95 # runs training.gen0.harness, pushes weights/<version>/):
96 RALLY_DETECTOR_IMPL=native python -m backend.worker train \
97     --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-rally-corpus \
98     --version <v> --trajectory-source detector --segmenter wasb_hybrid --config
config.json
99     ```
100
101 A bare/relative path resolves under a configured `input_root`/`input_backend`
102 (`backend/storage/ref.py::resolve_input_uri`), so workflows can be written bucket-
relative.
103
104 ---
105
106 ## 5. The gap ? what's still local-only (why the box is still a blocker)
107
108 All of this is tiny except the originals ? the metadata that blocks cloud eval is <
25 MB total:
109
110 | Data | Where it is now | Size | In bucket? | Needed for |
111 |---|---|---|---|---|
112 | 20 WASB trajectory CSVs | `C:\?\Annotation Setup\Collect\Trajectories\` | **17.5 MB**
| ? none | eval/litmus (cached), train (`--trajectory-source cached`) |
113 | golden manifest | `output/human_lovo_manifest.json` (C:-absolute paths) | 4.5 KB | ? |
eval corpus definition |
114 | original-6 labels | `F:[Khelsutra]?\Training\Golden Labelled\*.rallies.csv` | ~12 KB
| ? | train + eval |
115 | 7 new-clip labels | repo `output/*.rallies.csv` (gitignored) **** F: archive (backed
up 2026-06-22) | ~12 KB | ? partial/inconsistent | train + eval |
116 | source videos / proxies | F: archive + Google Drive golden folder | **100s of GB** | ?
3 only | infer + train (real WASB extraction) |
117
118 > The corpus *metadata* (trajectories + labels + manifest, ~25 MB) is what actually
blocks GPU-free
119 > cloud eval and the served Gate-A litmus ? and it's trivially small to migrate. The
**videos** are the
120 > only large migration, and they already exist on **Google Drive** (the golden-corpus
folder), so they
121 > can go **Drive ? bucket from a cloud box**, keeping the local machine out of the path
entirely.
122
123 ---
124
125 ## 6. Migration plan (close the gap)
126
127 **Phase 1 ? corpus metadata (?25 MB, do first, removes the eval blocker).** Stage all
trajectories,
128 all labels, and a **bucket-relative eval manifest** into `gs://khelsutra-rally-corpus`.
Pending the §7
129 naming decision. Sketch:
130
131 ```bash
132 # trajectories (decision-free; absent today)
133 gcloud storage cp "C:\?\Collect\Trajectories\*.csv" gs://khelsutra-rally-
corpus/trajectories/
134 # labels (after the §7.1 naming decision) -> labels/<canonical-name>.rallies.csv
135 # eval manifest with gs:// (or bucket-relative) paths -> a stable manifest object (§7.3)
136 ```

```

```

137
138     **Phase 2 ? videos ? `videos/<name>.mp4`** (bucket name = the eval `name`, consistent
with
139     `trajectories/` + `labels/`). Sources (resolved 2026-06-22 ? see
`scratch/copy_videos_to_gcs.ps1`):
140
141     | Source | Clips | Size |
142     |---|---|---|
143     | Drive golden folder `~/Golden Label Videos Request - June 15/[Done] Video N/`
(**proxies**) | `mahadevpura_1/2`, `GX010128`, `GX020094`, `GX030094`, `Badminton_BXH_2`,
`Boxhill_Doubles` (7) | ~13.5 GB |
144     | F: archive (**full MP4s** ? proxy-first before upload) | `GX010137`, `GX010141`, +
original-6 (`adarsh_avi_singles`, `kushagra_singles`, `largetest_doubles`, `gbaaddy`,
`testlarge_short`, `mahadevpura_singles`) (8) | ~50 GB |
145
146     **Two ways to run it:**
147     - **Decoupled (preferred ? no local box):** `rclone` on a cloud VM with a Google-Drive
remote ?
148     `rclone copy gdrive:"<?/[Done] Video N>/<proxy>.mp4" :gcs:khelsutra-rally-
corpus/videos/<name>.mp4`
149     (one-time owner step: `rclone config` ? `gdrive` remote OAuth). Or `gdown <file-id>`
on the box for
150     the shared folder. This is the path that actually removes the local box.
151     - **Interim (today, owner box ? streams Drive/F: ? local ? GCS):** `pwsh
scratch/copy_videos_to_gcs.ps1
152     -Execute` (dry-run without `-Execute`; `-Only <name>` for one; skip-existing). Uses
local bandwidth.
153
154     **Priority:** the 7 Drive **proxies** first (already 1080p, sufficient for detection).
The 8 large F:
155     full MP4s should be **proxied to 1080p first** (cheaper storage + faster cloud decode)
before upload.
156     ? Two pre-existing non-canonical video objects (`videos/GX030094_proxy.mp4`,
`videos/mahadevpura-1.MP4`)
157     are superseded once `videos/GX030094.mp4` / `videos/mahadevpura_1.mp4` land ? verify-
dupe-then-delete
158     (same as the label cleanup). Track per-clip in $8.
159
160     **Phase 3 ? flip eval/training to read GCS by default.** Point the eval manifest +
litmus at the
161     bucket (env override or `input_root`) so a fresh box/CI needs no local corpus. Re-verify
the suite +
162     the served Gate-A litmus read from `gcs://` (and re-measure if the corpus set changes ?
see PR #236).
163
164     ---
165
166     ## 7. Decisions
167
168     1. **`labels/` naming ? LOCKED (owner, 2026-06-22):** `labels/<YYYY-MM-
DD>_<name>.rallies.csv`, where
169     `<name>` is the eval/manifest clip name and `<YYYY-MM-DD>` is the label's **authoring
date** =
170     the *earliest* mtime across all known copies (C: working copy + F: archive ? robust
to copy-resets;
171     a restored C: copy shows today, the F: archive keeps the real date). Fixes the
`_proxy` inconsistency
172     and disambiguates recycled GoPro names by date. *(Future, when videos are in the
bucket: optionally
173     add a `video_id`/MD5 sidecar for the strongest keying ? deferred, needs hashing each
video.)*
174     2. **Cache `trajectories/` ? YES, done 2026-06-22.** `trajectories/<name>.csv` cached so
cloud/CI eval +
175     the served Gate-A litmus run **GPU-free** (training can still regenerate via
`--trajectory-source detector`).

```

```

176 3. **Eval manifest in GCS ? done 2026-06-22.** `gs://khelsutra-rally-
corpus/manifests/golden.json`
177 (15 clips, `gs://` trajectory + label paths). The **local** litmus keeps its local-
path manifest;
178 flipping the litmus to read the GCS manifest is Phase 3.
179 4. **Originals migration scope ? OPEN** ? proxies-only vs full 4K, and how much of the
15-clip golden
180 set vs the wider archive. Source = Google Drive ? bucket from a cloud box (not the
local box).
181
182 > Phase-1 was run via a dry-run planner (resolve every clip's trajectory + label, stamp
the label's
183 > authoring date, build the `gs://` manifest) ? `gcloud storage cp` (planner kept in
gitignored
184 > `scratch/`; promote it to `scripts/` if corpus-sync recurs).
185 > ? **Cleanup DONE 2026-06-22:** the 7 pre-existing inconsistent label objects
(`*_proxy.rallies.csv`,
186 > bare `GX010128`/`mahadevpura-1`) were confirmed **byte-identical (MD5)** to the new
canonical names
187 > (content also on C:/F:) and **deleted** ? `labels/` now holds exactly the 15 canonical
objects.
188
189 > ? **gcloud gotcha (Phase-2):** `gcloud storage cp` treats `[...]` as a glob, so source
paths under
190 > `[Done] Video N/` (Drive) or `[Khelsutra] ?` (F:) fail with "matched no objects." The
owner-box
191 > script (`scratch/copy_videos_to_gcs.ps1`) stages such paths through a bracket-free
temp file; the
192 > `rclone` cloud path has no such issue.
193
194 ---
195
196 ## 8. Progress tracker
197
198 | Item | Status |
199 |---|---|
200 | Bucket + SA + provisioning (deploy/gcp) | ? built |
201 | Cloud worker `infer`/`train` by URI | ? built (`backend/worker`) |
202 | This data-map doc | ? 2026-06-22 |
203 | Phase 1: trajectories ? `trajectories/` (15) | ? 2026-06-22 |
204 | Phase 1: labels reconciled ? `labels/<date>_<name>` (15) | ? 2026-06-22 |
205 | Phase 1: eval manifest ? `manifests/golden.json` | ? 2026-06-22 |
206 | Phase 1: remove 7 stale `*_proxy`/bare label objects | ? 2026-06-22 (MD5-verified
dups, deleted) |
207 | Phase 2: corpus videos/proxies ? `videos/<name>.mp4` | ? mapping + script done;
**1/15** demo'd (`mahadevpura_1`); 14 remain ? run `rclone` on a cloud box (preferred) or the
owner-box script |
208 | Phase 2: proxy the 8 large F: full-MP4 clips before upload | ? |
209 | Phase 3: eval/litmus default to the GCS manifest | ? |
210
211 **Definition of done:** a fresh GPU box (or CI) can run inference, the eval suite, and
Gen-0 training
212 reading **only** from `gs://khelsutra-rally-corpus` ? no `C:` / `F:` / local-Drive path
required.
213
214 ---
215
216 ## Related
217 - [`SETUP_NEW_MACHINE.md`](SETUP_NEW_MACHINE.md) ? spin up a box (storage backends in
$7).
218 - [`deploy/gcp/README.md`](../deploy/gcp/README.md) ? GCP GPU provisioning + train/infer
($5).
219 - [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) .
[`PER_VIDEO_WORKSPACE.md`](PER_VIDEO_WORKSPACE.md) ? locality tiers + `outputs/<video_id>/`.
220 - `docs/archives/past_projects/DECOUPLED_COMPUTE/` ? the cloud-engine design lineage.

```

4. user (2026-06-22T10:25:24.829Z)

```

53467c9 docs(data): record stale-label cleanup + set up Phase-2 video copy
6ae3503 docs(data): record Phase-1 GCS migration (trajectories + labels + manifest)
8884c9f docs(data): add DATA_IN_GCS.md ? the corpus source-of-truth map (GCS)
689c738 docs(pkg): mark P2c ? + the loop closed ? v1 LIGHT is feature-complete (#313)
293e5c0 feat(pkg): re-bundle backend/ui ? serve the P2c first-run wizard from indexer --app
(#312)
f8b2c3b feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run
default (gate 08) (#310)
0ed4cc6 fix(segmenter): fail the job on an unrunnable config instead of a silent 0-rally success
(Finding B) (#311)
a42c132 docs(pkg): track the batteries-included app plan (PACKAGING_PLAN.md) + index pointers
(#309)
de2bf14 feat(pkg): P2e ? lazy-torch so the LIGHT tier imports with NO torch installed (default-
OFF) (#308)
679e642 feat(pkg): P2d ? DB upgrade-safety (downgrade guard + backup-before-migrate + export)
(default-OFF) (#307)
f01dc85 feat(db): v7 ? record the UI locale per job (PMF analytics) + count_jobs_by_locale
(#306)
fa7ee3f feat(pkg): P2a ? single ffmpeg/ffprobe resolver (env-overridable; one fail-loud place)
(default-OFF) (#305)
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
ca34787 feat(pkg): P2b ? friendly `indexer --app` launcher (boot localhost server + open the
bundled UI) (#303)
4a06b8b feat(pkg): P1b ? bundle the KhelSutra SPA single-origin into the engine wheel (default-
OFF) (#302)

```

5. assistant (2026-06-22T10:25:29.178Z)

Let me examine the relevant code paths to assess risks deeper.

6. user (2026-06-22T10:25:30.133Z)

```

1      """Storage value objects + URI parsing for the pluggable storage layer.
2
3      A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5          <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6          local://<path>             -> local    (local:///abs or local://rel/p)
7          s3://<bucket>/<key>        -> s3       (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8          gcs://<bucket>/<key>      -> gcs      (gs:// accepted, normalized to gcs://)
9          gdrive://<path>           -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24

```

```

25     @dataclass(frozen=True)
26     class StorageStat:
27         """Lightweight object metadata (uniform across backends)."""
28         size: int
29         modified: Optional[datetime] = None
30         etag: Optional[str] = None
31         content_type: Optional[str] = None
32
33
34     @dataclass(frozen=True)
35     class StorageRef:
36         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
37         ``gdrive`` it is
38         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
39         Drive)."""
40         backend: str
41         bucket: str
42         key: str
43         uri: str
44
45         @property
46         def name(self) -> str:
47             return self.key.rstrip("/").rsplit("/", 1)[-1]
48
49     def parse_uri(uri: str) -> StorageRef:
50         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
51
52         A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
53         have no ``//``) is treated as a local filesystem path ? preserving today's
54         behaviour.
55
56         """
57         s = str(uri)
58         m = _SCHEME_RE.match(s)
59         if not m:
60             return StorageRef("local", "", s, f"local://{s}")
61         scheme = m.group(1).lower()
62         rest = s[m.end():]
63         if scheme == "gs":
64             scheme = "gcs"
65         if scheme in ("local", "gdrive"):
66             # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
67             Drive".
68             return StorageRef(scheme, "", rest, s)
69         if scheme in _KNOWN:
70             bucket, _, key = rest.partition("/")
71             return StorageRef(scheme, bucket, key, s)
72         raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
73
74     def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
75     -> str:
76         """Resolve a user-supplied input reference into a storage URI
77         (COMPUTE_DECOUPLED_SERVING M2).
78
79         Precedence (first match wins):
80         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
81         ``gdrive://``) ? as-is.
82         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
83         local).
84         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
85         (``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
86         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
87         ``bucket/x`` ?
88         ``gcs://bucket/x``).

```



```

81         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83     So the default (``input_backend="local"`, ``input_root=""`) is an identity
passthrough for every
84     path used today, preserving byte-for-byte behaviour.
85     """
86     s = str(raw)
87     if _SCHEME_RE.match(s):          # 1. explicit scheme wins
88         return s
89     # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90     #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91     #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
drive prefix "X:".
92     if s.startswith(("/", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93         return s
94     if input_root:                    # 3. join a relative name under the configured
root
95         return input_root.rstrip("/") + "/" + s.lstrip("/")
96     if input_backend and input_backend != "local":    # 4. prefix a bucket/key with the
declared backend
97         return f"{input_backend}:{s}"
98     return s                          # 5. bare/relative local path ? unchanged
99

```

7. user (2026-06-22T10:25:30.729Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local:``)
8     is an identity passthrough; ``s3:`` / ``gcs:`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """
16    from __future__ import annotations
17
18    import argparse
19    import csv
20    import json
21    import logging
22    import os
23    import sys
24    import tempfile
25    from typing import Any, List
26
27    from backend import storage
28    from backend.compute import ComputeJobSpec, get_compute_backend
29    from backend.config import (
30        StartupCheckError,
31        enforce_startup_checks,
32        load_config,
33        probe_cuda_available,

```

```

34 )
35 from backend.infrastructure.database import Database
36 from backend.pipeline.segmenters.base import segmenter_registry
37 from backend.pipeline.validators.base import registry as validator_registry
38 from backend.reporting import emit_indexer_report
39 from backend.results import Failure
40 from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
41 from backend.utils.hashing import compute_video_id
42
43 logger = logging.getLogger("backend.worker")
44
45
46 def _coerce(v: str) -> Any:
47     """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
48     low = v.lower()
49     if low in ("true", "false"):
50         return low == "true"
51     for cast in (int, float):
52         try:
53             return cast(v)
54         except ValueError:
55             pass
56     return v
57
58
59 def _apply_overrides(config: Any, sets: List[str]) -> None:
60     """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
61     for kv in sets or []:
62         key, sep, val = kv.partition("=")
63         if not sep:
64             raise ValueError(f"--set expects k=v, got {kv!r}")
65         parts = key.split(".")
66         obj = config
67         for p in parts[:-1]:
68             obj = getattr(obj, p)
69         setattr(obj, parts[-1], _coerce(val))
70
71
72 def _write_intervals_csv(segments: List[dict], path: str) -> None:
73     """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
74     artifact (same schema as the rally-annotator labels)."""
75     with open(path, "w", newline="") as f:
76         w = csv.writer(f)
77         w.writerow(["start", "end", "shot_count", "ending_reason"])
78         for s in segments:
79             w.writerow([
80                 f'{float(s.get("start_time", 0.0)):.3f}',
81                 f'{float(s.get("end_time", 0.0)):.3f}',
82                 s.get("shot_count", ""),
83                 s.get("ending_reason", s.get("shot_count_confidence", "")),
84             ])
85
86
87 def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
88     with open(path, "w") as f:
89         json.dump({
90             "video_id": video_id,
91             "status": status,
92             "segment_count": len(segments),
93             "segments": [{ "start": float(s.get("start_time", 0.0)),
94                           "end": float(s.get("end_time", 0.0)) } for s in segments],

```

```

95         }, f, indent=2)
96
97
98     def _run_startup_checks(config: Any) -> bool:
99         """M5 per-profile startup checks for the worker (the compute process, so it probes
100         CUDA
101         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
102         aborts
103         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
104         no-op
105         (returns True, ZERO work) when no deployment.profile is selected.
106
107         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
108         imports
109         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
110         delegated to a
111         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
112         path a true
113         no-op of work, not just of output."""
114         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
115         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
116         path
117         try:
118             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
119             return True
120         except StartupCheckError:
121             return False # already logged at ERROR by enforce_startup_checks
122
123     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
124         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
125         once the
126         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
127         dispatched
128         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
129         re-dispatch.
130
131         A remote job that never writes a completion marker (a failed/hung container) makes
132         the bucket
133         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
134         (remote dispatch
135         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
136         escape."""
137         from backend.infrastructure.execution_policy import PolicyTimeout
138
139         spec = ComputeJobSpec(
140             video_uri=args.video_uri, out_uri=args.out_uri,
141             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
142         )
143         try:
144             result = compute.run_infer(spec)
145         except PolicyTimeout as e:
146             logger.warning("[worker] remote compute did not complete (no marker within
147             budget): %s", e)
148             return 4
149         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
150                     result.video_id, result.returncode, result.outputs_uri)
151         return result.returncode
152
153     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
154     int,
155                             status: str, storage_cfg: dict, scratch: str) -> None:
156         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
157         for a remote

```

```

144     dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
145     return. """
146     try:
147         marker = {"video_id": video_id, "returncode": returncode,
148                  "segment_count": segment_count, "status": status}
149         local = os.path.join(scratch, "_done.json")
150         with open(local, "w", encoding="utf-8") as f:
151             json.dump(marker, f)
152         storage.put(local, done_uri, config=storage_cfg)
153         logger.info("[worker] wrote completion marker -> %s", done_uri)
154     except Exception as e: # never fail a good run because the marker push hiccuped
155         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
156
157     def cmd_infer(args: argparse.Namespace) -> int:
158         config = load_config(args.config) if args.config else load_config()
159         _apply_overrides(config, args.set)
160         if not _run_startup_checks(config):
161             return 2
162
163         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
164         # process.
165         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
166         # through to the
167         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
168         # (the
169         # dispatcher forces compute_target=local_gpu), so it never recursively re-
170         # dispatches.
171         compute = get_compute_backend(config)
172         if compute is not None:
173             return _dispatch_remote(compute, args)
174
175         storage_cfg = config.storage.model_dump()
176
177         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
178         os.makedirs(scratch, exist_ok=True)
179         config.output.output_dir = scratch
180
181         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
182         # to scratch.
183         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
184                                config=storage_cfg)
185         print(f"[worker] pulled {args.video_uri} -> {local_video}")
186
187         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
188         # byte-identical).
189         video_id = compute_video_id(local_video)
190
191         # 3. VALIDATE (same registry as the main CLI).
192         val_results, val_context = validator_registry.run_all_with_context(local_video,
193                                config)
194         failures = [r for r in val_results if not r.passed]
195         db = Database(os.path.join(scratch, config.output.db_name))
196         db.add_video(video_id, local_video, "failed" if failures else "processed",
197                    [r.model_dump() for r in val_results])
198
199         segments: List[dict] = []
200         segmenter_actual = None
201         segmentation_ran = False
202         status = "failed"
203         try:
204             if failures:
205                 print("[worker] validation FAILED: "
206                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
207                 return 2

```

```

200         seg_name = args.segmenter or config.indexing.default_segmenter
201         segmenter_cls = segmenter_registry.get(seg_name)
202         if not segmenter_cls:
203             print(f"[worker] unknown segmenter: {seg_name!r} ")
204             f"(have {list(segmenter_registry.list_available())})")
205             return 2
206         segmenter_actual = seg_name
207         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
208         segmentation_ran = True
209         if isinstance(result, Failure):
210             print(f"[worker] segmenter FAILED: {result.message}")
211             return 3
212         segments = result.value if hasattr(result, "value") else result
213         status = "success"
214         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
215         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
216         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
217         if not segments:
218             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
or getattr(config.indexing, "detector_impl", None) or
"auto")
219             logger.warning(
220                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
221                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
222                 "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
223                 "the WASB env produced no detections, or the input resolution differs
from the tuned "
224                 "proxy resolution. Re-run with -v for the native command + frame
counts.",
225                 video_id, segmenter_actual, detector_impl,
226             )
227         finally:
228             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
229             emit_indexer_report(
230                 db=db, video_id=video_id, video_path=local_video, config=config,
231                 val_results=val_results, val_context=val_context,
232                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
233                 was_reingest=False, segmentation_ran=segmentation_ran,
234             )
235         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
236         out_local = os.path.join(scratch, "outputs", video_id)
237         os.makedirs(out_local, exist_ok=True)
238         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
239         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
240         out_prefix = args.out_uri.rstrip("/")
241         pushed = []
242         for fn in sorted(os.listdir(out_local)):
243             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
244             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
245             pushed.append(uri)
246         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
247         for u in pushed:

```

```

252         print("    ", u)
253
254     # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
255     # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
256     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
257     # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
258     if getattr(args, "done_uri", None):
259         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
260     return 0
261
262
263     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
264     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
265     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
266
267
268     def _probe_video_fps_width(path: str):
269         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
270
271         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
272         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
273         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
274         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
275         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
276         rather than guesses.
277         """
278         import json
279         import subprocess
280         try:
281             out = subprocess.check_output(
282                 [ffprobe_bin(), "-v", "error", "-select_streams", "v:0",
283                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
284                 stderr=subprocess.DEVNULL).decode()
285             st = (json.loads(out).get("streams") or [{}])[0]
286             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
287             fps = float(num) / float(den or "1")
288             width = float(st.get("width") or 0)
289             if fps > 0 and width > 0:
290                 return fps, width
291         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
292             pass
293         return None
294
295
296     def _resolve_train_detector(args: argparse.Namespace, config: Any):
297         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
298         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
299         stub=CPU mock). Returns the runner, or prints an error + returns None.
300
301         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the

```

```

302     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
303     no shuttle detector and is rejected.
304     """
305     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
306
307     seg_cls = segmenter_registry.get(args.segmenter)
308     if not seg_cls:
309         print(f"[worker] unknown segmenter: {args.segmenter!r} "
310               f"(have {list(segmenter_registry.list_available())})")
311         return None
312     seg = seg_cls(None, config)
313     if not isinstance(seg, TrajectoryHybridSegmenter):
314         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
315               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
316         return None
317     try:
318         runner, _ = seg._resolve_runner(config.get("indexing", {}))
319     except NotImplementedError as e:
320         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
321         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
322         print(f"[worker] detector not available: {e}")
323         return None
324     ok, msg = runner.healthcheck()
325     if not ok:
326         print(f"[worker] detector environment not ready: {msg}")
327         return None
328     print(f"[worker] detector ready ({args.segmenter}): {msg}")
329     return runner
330
331
332     def cmd_train(args: argparse.Namespace) -> int:
333         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
334         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
335
336         Two trajectory sources (the train pipeline + harness are identical either way):
337         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
338         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
339         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
340         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
341         is the M3.5 "first real training" path; only the trajectory source flips.
342         """
343         import subprocess
344
345         config = load_config(args.config) if args.config else load_config()
346         _apply_overrides(config, args.set)
347         if not _run_startup_checks(config):
348             return 2
349         storage_cfg = config.storage.model_dump()
350
351         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
352         labels_dir = os.path.join(scratch, "labels")
353         traj_dir = os.path.join(scratch, "trajectories")
354         videos_dir = os.path.join(scratch, "videos")
355         os.makedirs(labels_dir, exist_ok=True)
356         os.makedirs(traj_dir, exist_ok=True)
357
358         corpus = args.corpus_uri.rstrip("/")

```

```

359         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
360                               if u.lower().endswith(".csv"))
361         if len(label_uris) < 2:
362             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
363                   f"under {corpus}/labels/")
364         return 2
365
366         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
367         runner = None
368         video_by_stem: dict = {}
369         if args.trajectory_source == "detector":
370             os.makedirs(videos_dir, exist_ok=True)
371             runner = _resolve_train_detector(args, config)
372             if runner is None:
373                 return 2
374             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
375                 vname = storage.parse_uri(vuri).name
376                 if not vname.lower().endswith(_VIDEO_EXTS):
377                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
378                 video_by_stem[os.path.splitext(vname)[0]] = vuri
379
380             print(f"[worker] trajectory source: {args.trajectory_source}")
381             specs: List[dict] = []
382             for uri in label_uris:
383                 fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
384                 name = fname.replace(".rallies.csv", "").replace(".csv", "")
385                 local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
386                 if args.trajectory_source == "detector":
387                     vuri = video_by_stem.get(name)
388                     if not vuri:
389                         print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
390                         continue
391                     local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
392                                               config=storage_cfg)
393                     video_id = compute_video_id(local_video)
394                     # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
395                     # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
396                     meta = _probe_video_fps_width(local_video)
397                     if meta is None:
398                         print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
399                         continue
400                     fps, frame_width = meta
401                     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
402                     if not traj:
403                         print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
404                         continue
405                     specs.append({"name": name, "trajectory": traj, "labels": local_label,
406                                "fps": fps, "frame_width": frame_width})
407             else:
408                 from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
409                 traj = os.path.join(traj_dir, f"{name}_traj.csv")
410                 synth_trajectory_from_labels(local_label, traj, fps=args.fps,

```



```

frame_width=args.frame_width)
411         specs.append({"name": name, "trajectory": traj, "labels": local_label,
412                        "fps": args.fps, "frame_width": args.frame_width})
413
414     if len(specs) < 2:
415         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
416         return 2
417     manifest_path = os.path.join(scratch, "manifest.json")
418     with open(manifest_path, "w", encoding="utf-8") as f:
419         json.dump(specs, f, indent=2)
420     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
421     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
422
423     out_dir = os.path.join(scratch, "artifacts")
424     cmd = [sys.executable, "-m", "training.gen0.harness",
425            "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
426     if args.floor:
427         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
428                        if "://" in args.floor else args.floor)
429         cmd += ["--floor", floor_local]
430     print("[worker] training:", " ".join(cmd))
431     rc = subprocess.run(cmd).returncode
432     if rc != 0:
433         print(f"[worker] gen0 harness failed (rc={rc})")
434         return rc
435
436     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
437     bundle = os.path.join(out_dir, args.version)
438     out_prefix = args.out_uri.rstrip("/")
439     pushed = []
440     for fn in sorted(os.listdir(bundle)):
441         uri = f"{out_prefix}/weights/{args.version}/{fn}"
442         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
443         pushed.append(uri)
444     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
445     for u in pushed:
446         print("    ", u)
447     return 0
448
449
450 def build_parser() -> argparse.ArgumentParser:
451     p = argparse.ArgumentParser(prog="python -m backend.worker",
452                                description="Storage-aware worker: pull ? run pipeline ?
push.")
453     p.add_argument("-v", "--verbose", action="store_true",
454                   help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
455     sub = p.add_subparsers(dest="cmd", required=True)
456
457     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
458     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
459     pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
460     pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
461     pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
462     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
463     pi.add_argument("--max-frames", type=int, default=None)
464     pi.add_argument("--done-uri", default=None,

```

```

465             help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
466             "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
467             "Default-OFF: omit it for the normal local run.")
468     pi.add_argument("--set", action="append", default=[], metavar="k=v",
469                     help="config override, e.g. --set indexing.skip_ai_handoff=true")
470     pi.set_defaults(func=cmd_infer)
471
472     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
473     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
474                     "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
475     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
476     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
477     pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
478                     help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
479                     "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
480                     "(needs videos/ in the corpus; honours detector_impl).")
481     pt.add_argument("--segmenter", default="wasb_hybrid",
482                     help="trajectory hybrid whose detector is used for --trajectory-
source detector")
483     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
484     pt.add_argument("--config", default=None)
485     pt.add_argument("--scratch", default=None)
486     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
487     pt.add_argument("--frame-width", type=float, default=1280.0,
488                     help="synth trajectory frame width (detector probes the video)")
489     pt.add_argument("--set", action="append", default=[], metavar="k=v")
490     pt.set_defaults(func=cmd_train)
491     return p
492
493
494     def _setup_logging(verbose: bool) -> None:
495         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
496         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
497         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
498         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
499         entry), not at import, so importing the module never reconfigures a caller's
logging."""
500         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
501         # configured logging at import), which would leave the level at WARNING and re-
suppress the
502         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
503         logging.basicConfig(
504             level=logging.DEBUG if verbose else logging.INFO,
505             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
506             datefmt="%H:%M:%S",
507             force=True,
508         )
509
510

```

```

511     def main(argv=None) -> int:
512         args = build_parser().parse_args(argv)
513         _setup_logging(getattr(args, "verbose", False))
514         return args.func(args)
515
516
517     if __name__ == "__main__":
518         sys.exit(main())
519

```

8. assistant (2026-06-22T10:25:41.688Z)

I found a critical mismatch. Let me verify the manifest format expected by eval consumers vs what the doc says was written.

The key finding: `cmd\_train` builds the manifest as a ****JSON list of `{name, trajectory, labels, fps, frame\_width}` dicts**** (line 417-419), but the doc says `manifests/golden.json` has 15 entries with `gs://` trajectory + label refs. Let me check what the eval/litmus consumers actually expect.

9. user (2026-06-22T10:25:42.046Z)

Found 29 files

- tests\test_p2b_install.py
- scripts\uninstall.py
- scripts\install.py
- backend\worker__main__.py
- training\gen0\harness.py
- tests\test_worker.py
- tests\test_training_gen0.py
- backend\pipeline\detectors\wasb_infer.py
- backend\eval\rally_seg_eval.py
- tests\test_rally_seg_eval_report.py
- backend\eval\golden_fixtures.py
- backend\eval\fusion_audio.py
- tests\test_served_gate_a_regression.py
- tests\test_nightly_regression.py
- tests\test_fusion_compare.py
- scripts\run_phase3_eval.py
- scripts\nightly_regression.py
- docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py
- tests\test_golden_real_fixtures.py
- tests\test_golden_fixtures.py
- tests\test_splits.py
- backend\eval\splits.py
- backend\eval\serve_contrast.py
- backend\eval\distill_local.py
- backend\eval\served_gate_a.py
- backend\eval\fusion_golden.py
- backend\eval\batch.py
- backend\eval\calibrate_local.py
- backend\eval\rally_seq_proto.py

10. user (2026-06-22T10:25:42.147Z)

Found 40 files

- docs\DATA_IN_GCS.md
- docs\README.md
- tests\test_config.py
- docs\NEXT_STEPS.md
- backend\pipeline\segmenters\trajectory_hybrid.py
- docs\data_pipeline\GOLDEN_DATA_SHARING.md
- docs\RALLY_DETECTION_QUALITY_REPORT.md
- docs\PERSONALIZATION_PLAN.md

docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\PROMINENT_COURT_DETECTION.md
docs\GOLDEN_REGRESSION_FIXTURES.md
docs\ABLATION_LAYER.md
docs\ANALYZERS_AND_RECONCILERS.md
docs\RALLY_QUALITY_RESEARCH.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\QUALITY_ITERATIONS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
backend\pipeline\segmenters\fusion_hybrid.py
backend\eval\rally_seg_eval.py
tests\test_rally_seg_eval.py
tests\test_ablation.py
tests\test_experiment.py
backend\eval\ablation.py
tests\test_served_gate_a_regression.py
backend\eval\ablation_pdf.py
docs\archives\quality-iterations\README.md
tests\test_fusion_hybrid.py
backend\eval\windowing.py
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\archives\HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md
tests\test_serve_contrast.py
tests\test_promotion.py
backend\eval\serve_contrast.py
backend\eval\promotion.py
eval_baselines\experiment_leaderboard.json
backend\eval\served_gate_a.py
backend\eval\per_user_gate.py
tests\test_per_user_gate.py
tests\test_experiment_honest.py
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md

11. user (2026-06-22T10:25:46.849Z)

```

1      """SERVED Gate-A verification: run the *production* ``FusionSegmenter`` decision seams
2      on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.
3
4      The offline litmus (``backend.eval.ablation`` / ``fusion_compare``) measures Gate A on
the
5      **permissive proposal windowing** (``distill_local.PROPOSAL`` = recall-oriented) re-
scored
6      through the experiment harness's ``_gate_mask``. That validates the *signal*. It does
NOT
7      exercise the **served path**: in production the windows come from
8      ``trajectory_to_action_windows`` at the production operating point
9      (``velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0``),
the
10     net-crossing count is computed by ``FusionSegmenter._enrich_candidate_stats``, and the
gate is
11     ``FusionSegmenter._select_for_handoff``. THIS module drives exactly those production
methods ?
12     no reimplementations of the windowing or the gate ? on a cached trajectory CSV, so the
13     verification reflects what ``process_video`` actually hands to Gemini.
14
15     GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that
WOULD
16     be sent to the AI). With the person cue OFF (the default) nothing here loads torch or
touches
17     the GPU ? it just re-scores cached WASB trajectories.
18
19     Run (owner box, golden trajectories present):
20     python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
21     """
22     from __future__ import annotations

```

```

23
24 import argparse
25 import json
26 from dataclasses import dataclass
27 from typing import Any, Dict, List, Optional, Tuple
28
29 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30 from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
31 from backend.eval.calibrate_wasb import load_golden_gts
32 from backend.eval.metrics import Interval, score, Score
33 from backend.eval import segmentation_metrics as _segm
34
35 # Production windowing operating point (config.json indexing defaults ? the values
36 # TrajectoryHybridSegmenter.process_video reads for the served fusion run).
37 PROD_VELOCITY_THRESH = 0.02
38 PROD_MERGE_GAP = 2.0
39 PROD_MIN_WINDOW = 2.0
40
41
42 def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
43     """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
44     config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
45     person cue OFF (so no GPU / torch is touched)."""
46     return {
47         "fusion": {
48             "substrate": "wasb",
49             "min_crossings": min_crossings,
50             "min_players": 0,
51             "cue_flags": {},
52         },
53         "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
54         "dead_time_merge_gap": PROD_MERGE_GAP,
55         "min_action_window_duration": PROD_MIN_WINDOW,
56     }
57
58
59 def served_handoff_windows(trajecory_csv: str, fps: float, frame_width: float,
60                             min_crossings: int) -> List[Interval]:
61     """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
62     return the
63     windows that would reach the AI handoff (i.e. the served rally candidates).
64     Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
65     (computes
66     ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
67     production
68     ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
69     is OFF.
70     """
71     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
72     windows = TrackNetRunner.trajectory_to_action_windows(
73         points, fps=fps, frame_width=frame_width,
74         velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
75         min_window_duration=PROD_MIN_WINDOW,
76     )
77     idx_cfg = _served_idx_cfg(min_crossings)
78     # Construct the real served segmenter (db=None, config carries the indexing block).
79     We only
80     # call the pure decision seams ? no process_video, no DB, no provider ? exactly as
81     the
82     # registry constructs it for `name`/`description`.
83     seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
84     # The served per-window stats (incl. net_crossings) via the production enrichment
85     hook.
86     window_stats = [

```

```

81         seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
82                                     idx_cfg=idx_cfg)
83     for cw in windows
84 ]
85 keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
86 return [(windows[i]["start"], windows[i]["end"]) for i in keep]
87
88
89 @dataclass
90 class ServedVideoResult:
91     name: str
92     num_gt: int
93     on: Score          # Gate-A ON (min_crossings=2)
94     off: Score         # Gate-A OFF (min_crossings=0, control)
95     seg_ratio_on: float # |preds| / |gts| with Gate-A ON (over-segmentation)
96     seg_ratio_off: float
97     n_handoff_on: int
98     n_handoff_off: int
99
100     @property
101     def delta_f1(self) -> float:
102         return self.on.f1 - self.off.f1
103
104
105 def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
106                  on_crossings: int = 2) -> ServedVideoResult:
107     """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
108     windows for
109     one golden video against its golden labels."""
110     fps, fw = float(spec["fps"]), float(spec["frame_width"])
111     traj, labels = str(spec["trajectory"]), str(spec["labels"])
112     gts = load_golden_gts(labels)
113     preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
114     preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
115     return ServedVideoResult(
116         name=str(spec["name"]),
117         num_gt=len(gts),
118         on=score(preds_on, gts, iou),
119         off=score(preds_off, gts, iou),
120         seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
121         seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
122         n_handoff_on=len(preds_on),
123         n_handoff_off=len(preds_off),
124     )
125
126 def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
127                    on_crossings: int = 2) -> List[ServedVideoResult]:
128     return [evaluate_video(s, iou, on_crossings) for s in specs]
129
130
131 def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
132     """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
133     n = len(results)
134
135     def _mean(getter) -> float:
136         return sum(getter(r) for r in results) / n if n else 0.0
137
138     deltas = [r.delta_f1 for r in results]
139     wins = sum(1 for d in deltas if d > 1e-9)
140     losses = sum(1 for d in deltas if d < -1e-9)
141     return {
142         "num_videos": n,
143         "mean_f1_on": _mean(lambda r: r.on.f1),
144         "mean_f1_off": _mean(lambda r: r.off.f1),

```

```

145         "mean_delta_f1": _mean(lambda r: r.delta_f1),
146         "mean_precision_on": _mean(lambda r: r.on.precision),
147         "mean_precision_off": _mean(lambda r: r.off.precision),
148         "mean_recall_on": _mean(lambda r: r.on.recall),
149         "mean_recall_off": _mean(lambda r: r.off.recall),
150         "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
151         "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
152         "wins": wins,
153         "losses": losses,
154     }
155
156
157 def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) ->
str:
158     lines = [
159         f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou})
===",
160         f" {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
161         f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
162     ]
163     for r in results:
164         lines.append(
165             f" {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
166             f"{r.delta_f1:>7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "
167             f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}")
168     lines += [
169         f" {'-' * 92}",
170         f" MEAN F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on) "
171         f"?F1={agg['mean_delta_f1']:+.3f} "
172         f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f} "
173         f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
174         f" over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
175         f"{agg['mean_seg_ratio_on']:.2f} (on) | wins {agg['wins']}/{agg['num_videos']}"
176         f"{'losses {agg['losses']}'}",
177     ]
178     return "\n".join(lines)
179
180
181 def run(manifest_path: str, iou: float = 0.5) -> Tuple[List[ServedVideoResult],
Dict[str, Any]]:
182     with open(manifest_path, encoding="utf-8") as f:
183         specs = json.load(f)
184         results = evaluate_manifest(specs, iou=iou)
185         return results, aggregate(results)
186
187
188 def main(argv: Optional[List[str]] = None) -> None:
189     ap = argparse.ArgumentParser(
190         description="Verify served Gate-A on real footage via the production
FusionSegmenter path.")
191     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
192     ap.add_argument("--iou", type=float, default=0.5)
193     args = ap.parse_args(argv)
194     results, agg = run(args.manifest, iou=args.iou)
195     print(format_table(results, agg, args.iou))
196
197
198 if __name__ == "__main__":
199     main()
200

```

12. user (2026-06-22T10:25:47.635Z)

```

1     """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.

```

```

2
3   `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
4   replay code path** used by both the freeze tool and the characterization gate, and it
5   deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6   variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7   across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8   only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9   merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10  LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11  `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12  path simply never calls into it.)
13
14  The replay reuses the already-shipped eval stack VERBATIM::
15
16      parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17      ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18      ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20  Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
21
22  * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
23  provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24  * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"``.
25  * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26  person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27  * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
28  passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29  named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30  silently churn fixtures.
31  * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32  the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33  takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34  has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35  ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36  tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38  §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39  suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40  frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41  that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42  field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44  Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45  parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is

```



```

46     NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47     the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48     never calls into it.
49     """
50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import re
56     import sys
57     from dataclasses import dataclass
58     from pathlib import Path
59     from typing import Any, Dict, List, Optional, Sequence, Tuple
60
61     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-----
62     from backend.eval import distill_local
63     from backend.eval.experiment import CONTROL, VideoCandidates, predict
64     from backend.eval.gt_loader import load_gt_intervals
65     from backend.eval.metrics import score
66     from backend.eval.regression import gt_hash
67
68     Interval = Tuple[float, float]
69
70     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
71     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
72     CURRENT_SCHEMA = 1
73
74     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
75     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
76     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
77
78     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
79     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
80     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
81     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
82     #: re-states these in its provenance; the replay reads them from there.
83     PINNED_VT = 0.02
84     PINNED_MERGE_GAP = 2.0
85     PINNED_MIN_WINDOW = 2.0
86     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
87     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``pinned_proposal``
88     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
89     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
90     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
91     PINNED_MAX_WINDOW = 0.0
92
93     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
94     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.

```

```

95     SHUTTLE_FEATURES: Tuple[str, ...] = (
96         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
97     )
98     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
99         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
100    )
101
102    #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
103    #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
104    FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
105
106    _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
107    _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
108
109    #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
110    #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
111    _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
112
113    #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
114    #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
115    #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
116    #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
117    _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")
118
119
120    # =====
121    # Synthetic trajectory generation (deterministic ? NO random)
122    # =====
123
124
125    @dataclass(frozen=True)
126    class RallySpec:
127        """One in-flight rally segment: the shuttle oscillates across the net midline.
128
129        ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
130        period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
131        net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
132        """
133
134        start_frame: int
135        n_frames: int
136        period: float = 30.0
137        amplitude: float = 400.0
138
139
140    @dataclass(frozen=True)
141    class GapSpec:
142        """Dead-time between rallies ? what separates one window from the next.
143
144        ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
145        ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
end an active run so the next rally forms a distinct window.
146        """
147
148
149        start_frame: int
150        n_frames: int
151        kind: str = "still" # "still" | "occluded"

```

```

152
153
154 @dataclass(frozen=True)
155 class FixtureSpec:
156     """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
157
158     ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
159     "labels.csv". For a synthetic fixture they line up with the rallies by construction,
160     which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
161     field-quality one (§6).
162     """
163
164     slug: str
165     description: str
166     fps: float
167     frame_width: float
168     frame_height: float
169     segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
170     gts: Tuple[Interval, ...]
171     net_midline: float = 640.0
172
173
174     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
175                       ) -> List[Tuple[int, int, float, float]]:
176         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
177         rows: List[Tuple[int, int, float, float]] = []
178         y_mid = frame_height / 2.0
179         for i in range(spec.n_frames):
180             x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
181             # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
182             y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
183             rows.append((spec.start_frame + i, 1, x, y))
184         return rows
185
186
187     def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
188                    ) -> List[Tuple[int, int, float, float]]:
189         """Dead-time rows: resting-visible (still) or absent (occluded)."""
190         rows: List[Tuple[int, int, float, float]] = []
191         y_mid = frame_height / 2.0
192         for i in range(spec.n_frames):
193             if spec.kind == "occluded":
194                 rows.append((spec.start_frame + i, 0, 0.0, 0.0))
195             elif spec.kind == "still":
196                 rows.append((spec.start_frame + i, 1, net_midline, y_mid))
197             else: # pragma: no cover - guarded by builder
198                 raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still' | 'occluded')")
199         return rows
200
201
202     def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
203         """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
scenario."""
204         rows: List[Tuple[int, int, float, float]] = []
205         for seg in spec.segments:
206             if isinstance(seg, RallySpec):
207                 rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
208             elif isinstance(seg, GapSpec):
209                 rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
210             else: # pragma: no cover - guarded by dataclass shape
211                 raise TypeError(f"unknown segment type {type(seg).__name__}")
212         rows.sort(key=lambda r: r[0])
213         return rows

```

```

214
215
216 def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
217     """Write trajectory rows as the TrackNet CSV (`Frame,Visibility,X,Y`), LF + 6dp
floats."""
218     lines = ["Frame,Visibility,X,Y"]
219     for frame, vis, x, y in rows:
220         lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
221     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
222
223
224 def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
225     """Write GT intervals as a `start,end` labels CSV (the strict gt_loader format),
LF."""
226     lines = ["start,end"]
227     for s, e in gts:
228         lines.append(f"{_fmt(s)},{_fmt(e)}")
229     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
230
231
232 # =====
233 # Built-in synthetic scenarios (the 3 committed slugs)
234 # =====
235
236
237 def _fr(seconds: float, fps: float) -> int:
238     return int(round(seconds * fps))
239
240
241 def builtin_specs() -> List[FixtureSpec]:
242     """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
243     windowing behaviour (one window / dead-time split / occlusion split)."""
244     fps = 30.0
245     fw, fh = 1280.0, 720.0
246     net = 640.0
247
248     # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
249     single = FixtureSpec(
250         slug="clean_single_rally",
251         description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
252         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
253         segments=(RallySpec(start_frame=0, n_frames=120),),
254         # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
255         gts=((0.0, 4.0),),
256     )
257
258     # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
259     # low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
260     r0 = RallySpec(start_frame=0, n_frames=120)
261     gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
262     r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
263     multi = FixtureSpec(
264         slug="multi_rally_with_deadtime",
265         description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
266         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
267         segments=(r0, gap_still, r1),
268         gts=((0.0, 4.0), (7.0, 11.0)),
269     )
270

```

```

271         # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
272         #     visibility break ? proves occlusion (not just low velocity) separates windows.
273         o0 = RallySpec(start_frame=0, n_frames=120)
274         gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
275         o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
276         occ = FixtureSpec(
277             slug="occlusion_break",
278             description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
279             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
280             segments=(o0, gap_occ, o1),
281             gts=((0.0, 4.0), (7.0, 11.0)),
282         )
283
284         return [single, multi, occ]
285
286
287     def _spec_by_slug() -> Dict[str, FixtureSpec]:
288         return {s.slug: s for s in builtin_specs()}
289
290
291     # =====
292     # The shared replay (CONTROL / pure-stdlib path)
293     # =====
294
295
296     def _fmt(x: float) -> str:
297         """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
298         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
299         if r == 0.0:
300             r = 0.0
301         return f"{r:.6f}"
302
303
304     def _round(x: float) -> float:
305         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
306         return 0.0 if r == 0.0 else r
307
308
309     def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
310         """The windowing proposal tuple ``(vt, mg, mwd)``, read from the fixture provenance
when present
311         (the pinned params travel with the fixture), else the module-level pinned defaults.
NEVER
312         ``windowing.SERVED``. NOTE: the provenance ``max_win`` (W1) is intentionally NOT
read here ?
313         ``build_candidates`` takes only this 3-tuple, so the replay characterizes the W1-OFF
path; the
314         pinned ``max_win`` is a recorded annotation, not a knob the gate applies (see
PINNED_MAX_WINDOW)."""
315         if prov and "windowing" in prov:
316             w = prov["windowing"]
317             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
318         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
319
320
321     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
322         """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.
323
324         Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
325         ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:

```

```

326
327         parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
328         cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
329
330     Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
331     control_segments:[{s,e}], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
332     merges overlaps; NO numpy is touched.
333     ""
334     fdir = _resolve_dir(slug_or_dir)
335     slug = fdir.name
336     traj_csv = fdir / "trajectory.csv"
337     labels_csv = fdir / "labels.csv"
338     prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
339
340     fps, fw = _fps_fw(prov, slug)
341     iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
342     proposal = _pinned_proposal(prov)
343
344     intervals, feature_rows = distill_local.build_candidates(
345         str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
346
347     # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
348     # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
349     # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
350     features: Dict[str, List[float]] = {
351         name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
352     }
353     gts = load_gt_intervals(str(labels_csv), strict=True)
354     vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
355
356     preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
357     sc = score(preds, gts, iou)
358
359     candidates: List[Dict[str, Any]] = []
360     for j, (s, e) in enumerate(intervals):
361         shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
362         # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
363         shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
364         candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
365
366     return {
367         "slug": slug,
368         "schema_version": CURRENT_SCHEMA,
369         "iou": _round(iou),
370         "candidates": candidates,
371         "control_segments": [[_round(s), _round(e)] for s, e in preds],
372         "score": {
373             "iou_threshold": _round(sc.iou_threshold),
374             "num_pred": int(sc.num_pred),
375             "num_gt": int(sc.num_gt),
376             "true_positives": int(sc.true_positives),
377             "false_positives": int(sc.false_positives),
378             "false_negatives": int(sc.false_negatives),
379             "precision": _round(sc.precision),
380             "recall": _round(sc.recall),

```

```

381         "f1": _round(sc.f1),
382         "mean_iou": _round(sc.mean_iou),
383         "mean_start_error": _round(sc.mean_start_error),
384         "mean_end_error": _round(sc.mean_end_error),
385     },
386     "gt_hash": gt_hash(gts),
387 }
388
389
390 # =====
391 # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
392 # =====
393
394
395 def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
396                 ) -> Dict[str, Any]:
397     """The committed provenance block ? pins the windowing + privacy/licensing tags
398     (§4c, §5)."""
399     desc = ""
400     if isinstance(spec_or_dir, FixtureSpec):
401         desc = spec_or_dir.description
402     return {
403         "slug": slug,
404         "schema_version": CURRENT_SCHEMA,
405         "kind": "synthetic",
406         "license": "synthetic",
407         "minors_free": True,
408         "description": desc,
409         "fps": _round(fps),
410         "frame_width": _round(fw),
411         "iou": _round(iou),
412         "windowing": {
413             "vt": PINNED_VT,
414             "mg": PINNED_MERGE_GAP,
415             "mwd": PINNED_MIN_WINDOW,
416             "max_win": PINNED_MAX_WINDOW,
417         },
418         "paths": {
419             "trajectory": "trajectory.csv",
420             "labels": "labels.csv",
421             "expected": "expected.json",
422         },
423         "note": (
424             "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
425             "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
426             (§6).",
427         ),
428     }
429
430 def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
431                   iou: float = 0.5) -> Dict[str, Any]:
432     """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
433     COMMITTED
434     ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
435     construction).
436
437     Idempotent: rewrites the same bytes given the same committed inputs. Requires the
438     trajectory
439     + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
440     """
441     fdir = fixtures_dir / slug
442     traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
443     if not traj_csv.is_file() or not labels_csv.is_file():
444         raise FileNotFoundError(

```

```

441         f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
442
443     # Provenance first (it pins the windowing the replay reads), then replay ? expected.
444     spec = _spec_by_slug().get(slug)
445     fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
446     prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
447     _write_json(fdir / "provenance.json", prov)
448
449     expected = replay_fixture(fdir)
450     _write_json(fdir / "expected.json", expected)
451     return expected
452
453
454     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
455         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
456         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
457         specs = builtin_specs()
458         fixtures_dir.mkdir(parents=True, exist_ok=True)
459         slugs: List[str] = []
460         for spec in specs:
461             fdir = fixtures_dir / spec.slug
462             fdir.mkdir(parents=True, exist_ok=True)
463             rows = make_synthetic_trajectory(spec)
464             write_trajectory_csv(rows, fdir / "trajectory.csv")
465             write_labels_csv(spec.gts, fdir / "labels.csv")
466             freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
467             slugs.append(spec.slug)
468         _write_manifest(specs, fixtures_dir)
469         return slugs
470
471
472     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
473                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
474         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
475         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
regression; the
476         mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
477         if all_slugs:
478             targets = [d.name for d in sorted(fixtures_dir.iterdir())
479                        if d.is_dir() and (d / "trajectory.csv").is_file()]
480         elif slug:
481             targets = [slug]
482         else:
483             raise ValueError("refresh_snapshot needs --slug S or --all")
484         for s in targets:
485             freeze_fixture(s, fixtures_dir=fixtures_dir)
486         return targets
487
488
489     # =====
490     # Manifest entry derivation (shared ? used by both the synthetic and real-fixture
writers)
491     # =====
492
493
494     def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
495         """A manifest row for a single fixture, derived from its provenance block."""
496         slug = prov["slug"]
497         return {
498             "slug": slug,

```



```

499         "fps": _round(float(prov["fps"])),
500         "frame_width": _round(float(prov["frame_width"])),
501         "windowing": dict(prov.get("windowing", {})),
502         "kind": prov.get("kind", "synthetic"),
503         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
504         "paths": {
505             "dir": slug,
506             "trajectory": f"{slug}/trajectory.csv",
507             "labels": f"{slug}/labels.csv",
508             "expected": f"{slug}/expected.json",
509             "provenance": f"{slug}/provenance.json",
510         },
511     }
512
513
514     def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
515         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
516         synthetic)."""
517         entries = load_manifest(fixtures_dir)
518         entries = [e for e in entries if e.get("slug") != prov["slug"]]
519         entries.append(_manifest_entry_for(prov))
520         entries.sort(key=lambda e: str(e.get("slug")))
521         _write_json(fixtures_dir / "manifest.json", entries)
522
523     # =====
524     # Manifest + loaders
525     # =====
526
527
528     def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
529         entries = [{
530             "slug": spec.slug,
531             "fps": _round(spec.fps),
532             "frame_width": _round(spec.frame_width),
533             "windowing": {
534                 "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
535                 "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
536             },
537             "kind": "synthetic",
538             "schema_version": CURRENT_SCHEMA,
539             "paths": {
540                 "dir": spec.slug,
541                 "trajectory": f"{spec.slug}/trajectory.csv",
542                 "labels": f"{spec.slug}/labels.csv",
543                 "expected": f"{spec.slug}/expected.json",
544                 "provenance": f"{spec.slug}/provenance.json",
545             },
546         } for spec in specs]
547         _write_json(fixtures_dir / "manifest.json", entries)
548
549
550     def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
551         """Load the fixtures manifest (returns [] if absent)."""
552         path = fixtures_dir / "manifest.json"
553         if not path.is_file():
554             return []
555         data = _read_json(path)
556         if not isinstance(data, list): # pragma: no cover - corrupt manifest
557             raise ValueError(f"{path}: manifest must be a JSON array")
558         return data
559
560
561     def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
562         """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses

```

```

a fixture
563     whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
the
564     ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
forward-version
565     bump in one is never masked by an in-range value in the other (red-team #7)."""
566     fdir = fixtures_dir / slug
567     prov = _read_json(fdir / "provenance.json")
568     expected = _read_json(fdir / "expected.json")
569     for which, blob in (("expected", expected), ("provenance", prov)):
570         sv = int(blob.get("schema_version", 0))
571         if sv > CURRENT_SCHEMA:
572             raise ValueError(
573                 f"[slug]: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "
574                 f"upgrade the harness")
575     return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
576
577
578     # =====
579     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
580     # =====
581
582
583     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
584         """Return a list of human-readable mismatch messages (empty == match).
585
586         Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
587         TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
588         equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
589         safe (the repo runs with core.autocrlf=true).
590
591         Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
592         per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
593         a fixed-field loop alone would ignore an added field (red-team #5)."""
594         out: List[str] = []
595
596         def exact(path: str, a: Any, b: Any) -> None:
597             if a != b:
598                 out.append(f"[path]: {a!r} != {b!r} (exact)")
599
600         def approx(path: str, a: Any, b: Any) -> None:
601             try:
602                 if abs(float(a) - float(b)) > _FLOAT_TOL:
603                     out.append(f"[path]: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
604             except (TypeError, ValueError):
605                 out.append(f"[path]: non-numeric float field {a!r} vs {b!r}")
606
607         # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
608         if set(recomputed) != set(committed):
609             out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")
610
611         exact("slug", recomputed.get("slug"), committed.get("slug"))
612         exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
613         exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
614         approx("iou", recomputed.get("iou"), committed.get("iou"))

```

```

615
616 rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
617 if len(rc) != len(cc):
618     out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
619 else:
620     for i, (r, c) in enumerate(zip(rc, cc)):
621         approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
622         approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
623         rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
624         if set(rs) != set(cs):
625             out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
626     else:
627         for k in rs:
628             if k == "net_crossings":
629                 exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
630             else:
631                 approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
632
633 rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
634 if len(rseg) != len(cseg):
635     out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
636 else:
637     for i, (r, c) in enumerate(zip(rseg, cseg)):
638         approx(f"control_segments[{i}][0]", r[0], c[0])
639         approx(f"control_segments[{i}][1]", r[1], c[1])
640
641 rsc, csc = recomputed.get("score", {}), committed.get("score", {})
642 # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
643 if set(rsc) != set(csc):
644     out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
645 int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
646 float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
"mean_start_error", "mean_end_error")
647
648 for k in int_fields:
649     exact(f"score.{k}", rsc.get(k), csc.get(k))
650 for k in float_fields:
651     approx(f"score.{k}", rsc.get(k), csc.get(k))
652
653 return out
654
655
656 # =====
657 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
658 # =====
659
660
661 def _write_json(path: Path, obj: Any) -> None:
662     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
663     path.write_text(text + "\n", encoding="utf-8", newline="\n")
664
665
666 def _read_json(path: Path) -> Any:
667     with open(path, encoding="utf-8") as f:
668         return json.load(f)
669
670
671 def _resolve_dir(slug_or_dir: Any) -> Path:
672     if isinstance(slug_or_dir, Path):
673         return slug_or_dir
674     p = Path(str(slug_or_dir))
675     if p.is_dir():

```

```

676         return p
677     return FIXTURES_DIR / str(slug_or_dir)
678
679
680 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
681     if prov and "fps" in prov and "frame_width" in prov:
682         return float(prov["fps"]), float(prov["frame_width"])
683     spec = _spec_by_slug().get(slug)
684     if spec is not None:
685         return spec.fps, spec.frame_width
686     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
687
688
689 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
690     prov_path = fdir / "provenance.json"
691     prov = _read_json(prov_path) if prov_path.is_file() else None
692     return _fps_fw(prov, slug)
693
694
695 # =====
696 # CLI
697 # =====
698
699
700 def _cmd_check(fixtures_dir: Path) -> int:
701     manifest = load_manifest(fixtures_dir)
702     if not manifest:
703         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
704         return 1
705     failures = 0
706     for entry in manifest:
707         slug = entry["slug"]
708         committed = _read_json(fixtures_dir / slug / "expected.json")
709         recomputed = replay_fixture(fixtures_dir / slug)
710         mismatches = compare_expected(recomputed, committed)
711         if mismatches:
712             failures += 1
713             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
f"(characterization = behaviour-locking, NOT correctness ? $6):")
714             for m in mismatches:
715                 print(f"    - {m}")
716         else:
717             sc = recomputed["score"]
718             print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
719     print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
"
f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
720     return 0 if failures == 0 else 1
721
722
723
724
725
726 def main(argv: Optional[Sequence[str]] = None) -> int:
727     ap = argparse.ArgumentParser(
728         description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
729     g = ap.add_mutually_exclusive_group(required=True)
730     g.add_argument("--freeze-synthetic", action="store_true",
731         help="generate synthetic trajectory+labels for all built-in
scenarios, then "
f"freeze expected.json+provenance.json + write manifest.json")
732     g.add_argument("--refresh-snapshot", action="store_true",
733         help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")

```

```

735         g.add_argument("--check", action="store_true",
736                         help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
737         g.add_argument("--freeze-real", action="store_true",
738                         help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
739                             "fixture; REFUSED without --owned --minors-free and a non-blank
--license")
740         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
741                             "--freeze-real (default: a random fx_<hex> id)")
742         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
743         ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
(review the diff!)")
744         # --freeze-real (P1 / M2 refusal gate) options:
745         ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
746         ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
(start,end)")
747         ap.add_argument("--license", dest="license_", default=None,
748                         help="--freeze-real) the source license tag (required, non-blank)")
749         ap.add_argument("--owned", action="store_true",
750                         help="--freeze-real) assert owner-recorded Fork-A source
(required)")
751         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
752                         help="--freeze-real) assert the source contains NO minors
(required)")
753         ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
754         ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
real) source frame width (px)")
755         ap.add_argument("--label", default="cleared_real",
756                         help="--freeze-real) non-identifying scenario label ? STRICT slug
only "
757                             "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
758         ap.add_argument("--source-note", dest="source_note", default="",
759                         help="--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
760                             "attribution back-channel, so it never reaches the committed
provenance "
761                             "(scanned defensively, then dropped). Retained for CLI
compatibility.")
762         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
763                         help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
764         ap.set_defaults(time_origin_reset=True)
765         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
766                         help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
767         args = ap.parse_args(list(argv) if argv is not None else None)
768
769         fixtures_dir: Path = args.fixtures_dir
770
771         if args.freeze_synthetic:
772             slugs = freeze_synthetic(fixtures_dir)
773             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
'.join(slugs)}")
774             print(f"  -> {fixtures_dir}")
775             return 0
776
777         if args.refresh_snapshot:
778             if not args.slug and not args.all:
779                 ap.error("--refresh-snapshot requires --slug S or --all")
780             # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a

```

```

real
781         # regression, so the deliberate, recorded reason is a required arg, not a soft
warning.
782         if not (args.reason and args.reason.strip()):
783             ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
stamp a real "
784                     "regression; record the deliberate reason and review the visible
diff ($6).")
785         targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
786         print(f"[golden-fixtures] refreshed snapshot for: {' ', '.join(targets)}")
787         f" (reason: {args.reason})")
788         return 0
789
790     if args.freeze_real:
791         # The real-fixture subsystem (P1 + M2) lives in golden_real_fixtures (split
out); import it
792         # here so the CLI reaches the SAME freeze_real_fixture / RefusalError it always
has.
793         from backend.eval import golden_real_fixtures
794         missing = [n for n, v in (("--trajectory", args.trajectory),
795                                 ("--labels", args.labels),
796                                 ("--fps", args.fps),
797                                 ("--frame-width", args.frame_width)) if v is None]
798         if missing:
799             ap.error("--freeze-real requires " + " ".join(missing))
800         try:
801             expected = golden_real_fixtures.freeze_real_fixture(
802                 args.label, args.trajectory, args.labels,
803                 license=args.license_ if args.license_ is not None else "",
804                 minors_free=args.minors_free, owned=args.owned,
805                 fps=args.fps, frame_width=args.frame_width,
806                 fixtures_dir=fixtures_dir, slug=args.slug,
807                 time_origin_reset=args.time_origin_reset, source_note=args.source_note)
808         except golden_real_fixtures.RefusalError as e:
809             # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
zero.
810             print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
811             return 2
812             print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
"
813                 f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
814             print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
815                 "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
816             return 0
817
818     # --check
819     return _cmd_check(fixtures_dir)
820
821
822     # =====
823     # Re-exports ? the real-fixture subsystem (P1 + M2) lives in ``golden_real_fixtures``
but
824     # is exposed here so every historical ``golden_fixtures.<name>`` import path stays
valid.
825     # =====
826
827     #: The names the real-fixture module owns; re-exported lazily (PEP 562 ``__getattr__``)
so the
828     #: ``golden_fixtures`` ? ``golden_real_fixtures`` cycle resolves regardless of import
order ?
829     #: ``golden_real_fixtures`` imports the shared primitives FROM this module, so an EAGER

```

```

bottom-of-
830     #: file re-export would fail when ``golden_real_fixtures`` is imported FIRST (its own
names would
831     #: not yet be defined when this module's import of them runs). The lazy hook defers the
import to
832     #: first attribute access, by which point BOTH modules are fully initialized.
833     _REAL_FIXTURE_EXPORTS: Tuple[str, ...] = (
834         "RefusalError", "freeze_real_fixture", "_scan_forbidden", "_new_surrogate_slug",
835         "_reset_trajectory_rows", "_reset_labels", "_SCHEMA_FILENAME_LITERALS",
836     )
837
838
839     def __getattr__(name: str) -> Any:
840         """Lazily resolve the re-exported real-fixture names from ``golden_real_fixtures``
(PEP 562)."""
841         if name in _REAL_FIXTURE_EXPORTS:
842             from backend.eval import golden_real_fixtures
843             return getattr(golden_real_fixtures, name)
844             raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
845
846
847     if __name__ == "__main__":
848         raise SystemExit(main())
849

```

13. assistant (2026-06-22T10:25:55.312Z)

Now I understand the manifest consumer. The ``served_gate_a.run()`` (line 181-185) does ``json.load(f)`` and expects a `**flat list of specs**` where each entry has ``name``, ``trajectory``, ``labels``, ``fps``, ``frame_width`` keys (read in ``evaluate_video`` line 109-112). Critically, `**`fps`` and ``frame_width`` are required keys**.

The doc says ``manifests/golden.json`` has 15 entries with ``gs://`` trajectory + label refs. Let me check whether the format the migration wrote includes ``fps`/`frame_width``, and check the label format consumed by ``load_golden_gts``. Let me also examine the existing local manifest and gt loaders.

14. user (2026-06-22T10:25:58.037Z)

```

[
  {
    "name": "adarsh_avi_singles",
    "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/adarsh_avi_full_traj.csv",
    "fps": 59.94,
    "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh and
Avi.rallies.csv"
  },
  {
    "name": "kushagra_singles",
    "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/kushagra_traj.csv",
    "fps": 59.94,
    "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/KushagraYashAviFirstVideo.rallies.csv"
  },
  {
    "name": "largetest_doubles",
    "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/largetestvideo_traj.csv",
    "fps": 29.97,
    "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden

```

```

Labelled/LargeTestVideo.rallies.csv"
},
{
  "name": "gbaaddy",
  "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/gbaaddy_traj.csv",
  "fps": 59.94,
  "frame_width": 1920,
  "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/gBaaddy20May2026.rallies.csv"
},
{
  "name": "testlarge_short",
  "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/testlarge_traj.csv",
  "fps": 29.97,
  "frame_width": 1920,
  "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/TestLargeVideo.rallies.csv"
},
{
  "name": "mahadevpura_singles",
  "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura_traj.csv",
  "fps": 29.97,
---SEARCH---
output/Badminton_BXH_2_golden_features.json
output/Boxhill_Doubles_golden_features.json
output/GX010128_golden_features.json
output/GX010137_golden_features.json
output/GX010141_golden_features.json
output/GX020094_golden_features.json
output/GX030094_golden_features.json
output/_staleness_fixes.json
output/ab_experiment_n6.json
output/adarsh_avi_singles_golden_features.json
output/cloud_lovo_manifest.json
output/gbaaddy_golden_features.json
output/gemini_GX010142_19june.json
output/gemini_GX010143_19june.json
output/gemini_GX010145_19june.json
output/human_lovo_manifest.json
output/kushagra_1280.report.json
output/kushagra_singles_golden_features.json
output/largetest_doubles_golden_features.json
output/local_calib_manifest.json
output/local_rally_model.json
output/local_vs_gemini_6.json
output/mahadevpura_1_golden_features.json
output/mahadevpura_2_golden_features.json
output/mahadevpura_singles_golden_features.json
output/mahadevpura_smoke_180s.report.json
output/person_GX010142.json
output/phase3_probe.json
output/testlarge_short_golden_features.json

```

15. user (2026-06-22T10:26:00.147Z)

```

commit 6ae3503d79f13b2f1e159005eaa18245a686fde4
Author: Avi Dullu <avi.dullu@gmail.com>
Date: Mon Jun 22 15:36:44 2026 +0530

```

docs(data): record Phase-1 GCS migration (trajectories + labels + manifest)

Phase-1 done 2026-06-22 ? corpus metadata now in gs://khelsutra-rally-corpus:

- trajectories/<name>.csv (15)
- labels/<YYYY-MM-DD>_<name>.rallies.csv (15) ? naming LOCKED (owner): manifest name + authoring-date prefix (earliest mtime across C:/F: copies, robust to copy-resets); fixes the _proxy inconsistency + disambiguates recycled GoPro names
- manifests/golden.json ? gs://-pathed eval manifest (15 clips)

Lock §7 decisions (naming / cache-trajectories / GCS manifest = done; originals scope still open), mark §8 Phase-1 rows ?, note the 7 stale pre-existing label objects to verify-and-delete.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

```
docs/DATA_IN_GCS.md | 42 ++++++-----
1 file changed, 32 insertions(+), 10 deletions(-)
===PHASE1 DIFF===
commit 6ae3503d79f13b2f1e159005eaa18245a686fde4
Author: Avi Dullu <avi.dullu@gmail.com>
Date: Mon Jun 22 15:36:44 2026 +0530
```

```
docs(data): record Phase-1 GCS migration (trajectories + labels + manifest)
===scratch tracked?===
scratch/analyze_motion.py
scratch/clean_state.py
scratch/diagnose_rally.py
scratch/gemini_video_debugger.py
scratch/list_models.py
scratch/test_ai_segmenter.py
scratch/test_thresholds.py
===gitignore scratch===
Binary file .gitignore matches
```

16. assistant (2026-06-22T10:26:05.419Z)

Important findings emerging. The local manifest carries per-clip `fps` (59.94, 29.97 ? varying) and `frame\_width` (1920). These are **load-bearing** for `served\_gate\_a` (wrong fps mis-aligns every label). Let me check whether the `name` keys in the GCS manifest match the eval clip names, the gt loader's CSV format expectation, and whether the cloud manifest the migration built includes fps/frame_width. Let me also check the scratch gitignore properly and the cloud_lovo_manifest.

17. user (2026-06-22T10:26:07.524Z)

```
===cloud_lovo_manifest (if this is the GCS one)===
[
  {
    "name": "adarsh_avi_singles",
    "fps": 59.94,
    "frame_width": 1920,
    "trajectory": "corpus/trajectories/adarsh_avi_full_traj.csv",
    "labels": "corpus/labels/Adarsh and Avi.rallies.csv"
  },
  {
    "name": "kushagra_singles",
    "fps": 59.94,
    "frame_width": 1920,
    "trajectory": "corpus/trajectories/kushagra_traj.csv",
    "labels": "corpus/labels/KushagraYashAviFirstVideo.rallies.csv"
  },
  {
    "name": "largetest_doubles",
    "fps": 29.97,
    "frame_width": 1920,
    "trajectory": "corpus/trajectories/largetestvideo_traj.csv",
    "labels": "corpus/labels/LargeTestVideo.rallies.csv"
  },
]
```

```

{
  "name": "gbaaddy",
  "fps": 59.94,
  "frame_width": 1920,
  "trajectory": "corpus/trajectories/gbaaddy_traj.csv",
  "labels": "corpus/labels/gBaaddy20May2026.rallies.csv"
},
{
  "name": "testlarge_short",
  "fps": 29.97,
  "frame_width": 1920,
  "trajectory": "corpus/trajectories/testlarge_traj.csv",
  "labels": "corpus/labels/TestLargeVideo.rallies.csv"
},
{
  "name": "mahadevpura_singles",
  "fps": 29.97,
  "frame_width": 1920,
  "trajectory": "corpus/trajectories/mahadevpura_traj.csv",
  "labels": "corpus/labels/MahadevpuraSingles.rallies.csv"
},
{
  "name": "mahadevpura_2",
  "fps": 59.94,
  "frame_width": 1920.0,
  "trajectory": "corpus/trajectories/mahadevpura2_june12_traj.csv",
  "labels": "corpus/labels/mahadevpura_2.rallies.csv"
},

```

===gitignore scratch line===

Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from scratch/

18. user (2026-06-22T10:26:08.360Z)

```

1      """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3      Wires the already-tested pieces together:
4          WASB trajectory CSV (Frame,Visibility,X,Y)
5          -> TrackNetRunner.trajectory_to_action_windows(cfg)    [predict_fn]
6          -> backend.eval.calibration (improvement_report / cross_validate)
7
8      Run (after a full-video WASB pass exists):
9          python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10             --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13      same GT is an **optimistic** upper bound. The trustworthy number is the
14      cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15      `leave_one_video_out` for precision/F1 generalization).
16      """
17      from __future__ import annotations
18
19      import argparse
20      from typing import List, Tuple, Callable
21
22      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
23      from backend.eval.calibration import (
24          select_best, improvement_report, cross_validate, evaluate,
25      )
26
27      Interval = Tuple[float, float]
28      Config = Tuple[float, float, float]    # (velocity_thresh, merge_gap,

```

```

min_window_duration)
29
30     BASELINE: Config = (0.02, 2.0, 2.0)    # the WasbConfig / trajectory_to_action_windows
defaults
31
32
33     def load_golden_gts(csv_path: str) -> List[Interval]:
34         """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
35
36         Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37         start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38         driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
39         """
40         from backend.eval.gt_loader import load_gt_intervals
41         return load_gt_intervals(csv_path, strict=False)
42
43
44     def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45         """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
47
48         def predict_fn(cfg: Config) -> List[Interval]:
49             vt, mg, mwd = cfg
50             wins = TrackNetRunner.trajectory_to_action_windows(
51                 points, fps=fps, frame_width=frame_width,
52                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
53             )
54             return [(w["start"], w["end"]) for w in wins]
55
56         return predict_fn
57
58
59     def default_grid() -> List[Config]:
60         return [(vt, mg, mwd)

```

19. assistant (2026-06-22T10:26:15.141Z)

Now I have a critical discovery. The `cloud\_lovo\_manifest.json` in `output/` (which appears to be a precursor to the GCS manifest) uses paths like `corpus/labels/Adarsh` and `Avi.rallies.csv` ? these are the ****OLD raw label filenames with spaces and original names****, NOT the new canonical `labels/<YYYY-MM-DD>\_<name>.rallies.csv` scheme that Phase-1 wrote to the bucket.

This is a potential dangling-reference risk. The CONTEXT says "every trajectory+label gs:// ref RESOLVES (0 dangling)" for `manifests/golden.json`, so the actual bucket manifest may differ from this `output/cloud\_lovo\_manifest.json`. But the label **trajectory** names also matter. Let me verify the trajectory naming and the gt_loader format, plus confirm the bracket gotcha and the deletion safety.

20. user (2026-06-22T10:26:16.514Z)

```

1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5       ``curate export`` format), raises on bad rows;
6     - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13

```

```

14 Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15 - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16   columns ? rally_number, ending_reason, sport, ? ? are ignored);
17 - no header: columns 0,1 positionally.
18 Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19 (``annotations.parse_timestamp``).
20 """
21 import csv
22 import logging
23 from typing import List, Optional, Tuple
24 from backend.eval.annotations import parse_timestamp
25
26 logger = logging.getLogger(__name__)
27
28 Interval = Tuple[float, float]
29
30 _START_NAMES = ("start", "start_time")
31 _END_NAMES = ("end", "end_time")
32
33
34 def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35     """If `row` is a header naming start/end columns, return their indices, else
36     None."""
37     cells = [(c or "").strip().lower() for c in row]
38     start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
39     end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
40     if start_idx is not None and end_idx is not None:
41         return start_idx, end_idx
42     return None
43
44 def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45     """Load GT rally intervals from either golden-CSV convention.
46
47     strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48     (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49     strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50     skipped ? a silently shrinking GT set flatters precision.
51
52     Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53     (they break one-to-one matching and usually indicate a labeling error).
54     """
55     intervals: List[Interval] = []
56     skipped = 0
57     cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
58     saw_header = False
59
60     with open(csv_path, newline="", encoding="utf-8-sig") as f:
61         for lineno, row in enumerate(csv.reader(f), start=1):
62             if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                 continue
64             if cols is None and not saw_header:
65                 header = _header_columns(row)
66                 if header is not None:
67                     cols = header
68                     saw_header = True
69                 continue # consume the header row
70             cols = (0, 1) # headerless: positional start,end

```

21. user (2026-06-22T10:26:18.218Z)

```

===trajectory names in cloud manifest vs bucket scheme===
"trajectory": "corpus/trajectories/adarsh_avi_full_traj.csv"
"trajectory": "corpus/trajectories/kushagra_traj.csv"

```

```

"trajectory": "corpus/trajectories/largetestvideo_traj.csv"
"trajectory": "corpus/trajectories/gbaaddy_traj.csv"
"trajectory": "corpus/trajectories/testlarge_traj.csv"
"trajectory": "corpus/trajectories/mahadevpura_traj.csv"
"trajectory": "corpus/trajectories/mahadevpura2_june12_traj.csv"
"trajectory": "corpus/trajectories/GX010128_june12_traj.csv"
"trajectory": "corpus/trajectories/mahadevpura1_june12_traj.csv"
"trajectory": "corpus/trajectories/GX030094_june15_traj.csv"
"trajectory": "corpus/trajectories/Badminton_BXH_2_june15_traj.csv"
"trajectory": "corpus/trajectories/GX010141_june17_traj.csv"
"trajectory": "corpus/trajectories/GX010137_june17_traj.csv"
"trajectory": "corpus/trajectories/Boxhill_Doubles_june18_traj.csv"
"trajectory": "corpus/trajectories/GX020094_june18_traj.csv"
===label names===
"labels": "corpus/labels/Adarsh and Avi.rallies.csv"
"labels": "corpus/labels/KushagraYashAviFirstVideo.rallies.csv"
"labels": "corpus/labels/LargeTestVideo.rallies.csv"
"labels": "corpus/labels/gBaaddy20May2026.rallies.csv"
"labels": "corpus/labels/TestLargeVideo.rallies.csv"
"labels": "corpus/labels/MahadevpuraSingles.rallies.csv"
"labels": "corpus/labels/mahadevpura_2.rallies.csv"
"labels": "corpus/labels/GX010128.rallies.csv"
"labels": "corpus/labels/mahadevpura_1.rallies.csv"
"labels": "corpus/labels/GX030094.rallies.csv"
"labels": "corpus/labels/Badminton_BXH_2.rallies.csv"
"labels": "corpus/labels/GX010141.rallies.csv"
"labels": "corpus/labels/GX010137.rallies.csv"
"labels": "corpus/labels/Boxhill_Doubles.rallies.csv"
"labels": "corpus/labels/GX020094.rallies.csv"

```

22. assistant (2026-06-22T10:26:27.338Z)

This confirms a major risk. The `output/cloud\_lovo\_manifest.json` (a draft/precursor that lives in the repo `output/`) uses **non-canonical names**: trajectories like `adarsh\_avi\_full\_traj.csv` (NOT `.csv`) and labels like `Adarsh and Avi.rallies.csv` (with spaces, original names ? NOT `

The CONTEXT asserts the **actual** bucket `manifests/golden.json` resolves with 0 dangling ? meaning the migration must have written a **different** manifest than this `output/cloud\_lovo\_manifest.json`. This drift between the repo-committed precursor manifest and the bucket manifest is itself a trap. Critically ? I cannot re-run gcloud to inspect the real bucket manifest, and the migration scripts that produced it live in gitignored `scratch/` (not reproducible).

Let me verify the key reproducibility gap and the bracket-glob gotcha by checking what scratch scripts exist and the trajectory naming convention (`<name>.csv` per doc vs `

23. user (2026-06-22T10:26:30.656Z)

```

===scratch scripts referenced in doc===
-rw-r--r-- 1 avidu 197609 4193 Jun 22 15:51 scratch/copy_videos_to_gcs.ps1

```

```

===any phase-1 planner in scratch?===
OWNED_MODEL_PHASE0_PLAN.md
boxhill_highlights_stage
copy_videos_to_gcs.ps1
golden.gcs.json
make_cloud_manifest.py
migrate_golden_to_gcs.py
stage
stage_corpus_to_gcs.sh

```

```

===full scratch listing===
ABLATION_FRAMEWORK_PLAN.md

```

GATE_AB_NUMBERS_AND_IMPLICATIONS.md
HIGH_RISK_REMEDIATION_PLAN.md
OWNED_MODEL_PHASE0_PLAN.md
PACKAGING_PLAN.serving-wip.md
PERSONALIZATION_FEATURE_DESIGN.md
PERSONALIZATION_PIVOT_ANALYSIS.md
PR1_BODY.md
PR1_HANDOFF.md
PR2_HANDOFF.md
PR2_deferred_hardening_status.patch
PR88_FIX.md
Rally_Detection_Quality_Report.pdf
__pycache__
_final_p1.png
_final_p4.png
_fusion_ablation_chart.png
_pov_final_p2.png
_pov_p1.png
_pov_p2.png
_rev_p1.png
_rev_p2.png
_rev_p3.png
_rev_p4.png
_rev_p5.png
_rev_p6.png
_rev_p7.png
_zoom_chart.png
_zoom_tbl1.png
ab_eval_demo.py
ab_n6_lovo.py
ablation_report.pdf
analyze_19june.py
analyze_motion.py
analyze_multishuttle_142.py
arial.ttf
at_par.py
boxhill_highlights_stage
ci_commit_msg.txt
clean_state.py
compare_19june_report.py
compare_local_vs_gemini.py
copy_videos_to_gcs.ps1
debug_wsl.py
diagnose_rally.py
eval_multitrack_19june.py
eval_multitrack_gx128_golden.py
extract_19june.py
extract_multishuttle_frames_142.py
extract_newclips_until_done.ps1
extract_person_19june.py
extract_until_done.ps1
extract_val_until_done.ps1
extract_wrapper.log
gate_ab_ablation.py
gate_ab_ablation_result.json
gate_ab_detail.py
gemini_detect_19june.py
gemini_long_detect.py
gemini_video_debugger.py
golden.gcs.json
gpu_validate
highlight_smoke_l4.mp4
hl_contact.png
keep_awake.ps1
keepawake.py

l4_intervals.csv
list_models.py
m2_deploy
make_boxhill_highlights.py
make_cloud_manifest.py
make_highlight.py
make_product_overview_pdf.py
migrate_golden_to_gcs.py
multitrack_select.py
native_smoke_l4.csv
obs_commit_msg.txt
opsagent.sh
overlay_tracks_142.py
p3_commit_msg.txt
p3_smoke.py
p4_commit_msg.txt
person_court_142.py
port.sh
probe_cv2_count.py
probe_gemini.py
quality_report.py
r1_evidence.py
r5_evidence.py
rally.tgz
reconstruct_prominent_track_142.py
repro_drawtext_font.py
repro_movie_overlay.py
run.sh
run_gx010125.py
run_gx010125_full.py
run_gx010125_rescore.py
run_wasb_gx128.py
segs
served_gateA_sweep.py
served_gateA_verify.py
sf_commit_msg.txt
speedup_commit_msg.txt
stage
stage_corpus_to_gcs.sh
stitch_min5.py
stitch_validate
test_ai_segmenter.py
test_thresholds.py
trim_video.py
upload_plan.tsv
val2_wrapper.log
val_wrapper.log
validate_multishuttle_142.py
verify_highlights.py
wasbfix.sh
wasbfix2.sh
watchdog_gx128.py

24. user (2026-06-22T10:26:32.660Z)

```
===is output/ gitignored? are these manifests committed?===  
output/cloud_lovo_manifest.json  
cloud_lovo IGNORED  
===git status output/===
```

25. assistant (2026-06-22T10:26:36.854Z)

The migration scripts DO exist locally (`scratch/migrate\_golden\_to\_gcs.py`,
`make\_cloud\_manifest.py`, `stage\_corpus\_to\_gcs.sh`, `golden.gcs.json`) but they're in
`\*gitignored scratch/\*` ? not in PR #316, not reproducible by anyone else. And

`output/cloud\_lovo\_manifest.json` is also gitignored. Let me read the actual migration script and the real bucket manifest snapshot to assess deletion safety and the manifest format that was actually pushed.

26. user (2026-06-22T10:26:38.438Z)

===golden.gcs.json (likely the ACTUAL pushed manifest)===

```
[
  {
    "name": "adarsh_avi_singles",
    "trajectory": "gs://khelsutra-rally-corpus/trajectories/adarsh_avi_singles.csv",
    "fps": 59.94,
    "frame_width": 1920,
    "labels": "gs://khelsutra-rally-corpus/labels/2026-06-08_adarsh_avi_singles.rallies.csv"
  },
  {
    "name": "kushagra_singles",
    "trajectory": "gs://khelsutra-rally-corpus/trajectories/kushagra_singles.csv",
    "fps": 59.94,
    "frame_width": 1920,
    "labels": "gs://khelsutra-rally-corpus/labels/2026-06-08_kushagra_singles.rallies.csv"
  },
  {
    "name": "largetest_doubles",
    "trajectory": "gs://khelsutra-rally-corpus/trajectories/largetest_doubles.csv",
    "fps": 29.97,
    "frame_width": 1920,
    "labels": "gs://khelsutra-rally-corpus/labels/2026-06-08_largetest_doubles.rallies.csv"
  },
  {
    "name": "gbaaddy",
    "trajectory": "gs://khelsutra-rally-corpus/trajectories/gbaaddy.csv",
    "fps": 59.94,
    "frame_width": 1920,
    "labels": "gs://khelsutra-rally-corpus/labels/2026-06-08_gbaaddy.rallies.csv"
  },
  {
    "name": "testlarge_short",
    "trajectory": "gs://khelsutra-rally-corpus/trajectories/testlarge_short.csv",
    "fps": 29.97,
    "frame_width": 1920,
    "labels": "gs://khelsutra-rally-corpus/labels/2026-06-08_testlarge_short.rallies.csv"
  },
  {
    "name": "mahadevpura_singles",
    "trajectory": "gs://khelsutra-rally-corpus/trajectories/mahadevpura_singles.csv",
    "fps": 29.97,
    "frame_width": 1920,
    "labels": "gs://khelsutra-rally-corpus/labels/2026-06-09_mahadevpura_singles.rallies.csv"
  },
  {
    "name": "mahadevpura_2",
    "trajectory": "gs://khelsutra-rally-corpus/trajectories/mahadevpura_2.csv",
    "fps": 59.94,
    "frame_width": 1920.0,
    "labels": "gs://khelsutra-rally-corpus/labels/2026-06-16_mahadevpura_2.rallies.csv"
  },
  {
    "name": "GX010128",
    "trajectory": "gs://khelsutra-rally-corpus/trajectories/GX010128.csv",
    "fps": 59.94,
    "frame_width": 1920.0,
    "labels": "gs://khelsutra-rally-corpus/labels/2026-06-16_GX010128.rallies.csv"
  },
  {

```



```
"name": "mahadevpura_1",
"trajectory": "gs://khelsutra-rally-corpus/trajectories/mahadevpura_1.csv",
```

27. user (2026-06-22T10:26:38.983Z)

```
===migrate_golden_to_gcs.py===
```

```
"""Plan (and optionally emit) the Phase-1 golden-corpus migration to GCS.
```

```
Naming (owner decision 2026-06-22): labels = `<YYYY-MM-DD>_<name>.rallies.csv` where the date is the
```

```
SOURCE label file's last-modified date (its authoring date). Trajectories = `<name>.csv` (manifest-name
```

```
keyed; the manifest links them). Builds a gs://-pathed eval manifest for cloud consumption.
```

```
Dry-run by default: prints the src->dst plan + writes the new manifest + an upload plan to scratch/.
```

```
Nothing is uploaded here ? the PowerShell step runs the verified `gcloud storage cp`s.
```

```
"""
```

```
import datetime
```

```
import json
```

```
import os
```

```
REPO = r"C:\Users\avidu\Projects\badminton-highlight-indexer"
```

```
MANIFEST = os.path.join(REPO, "output", "human_lovo_manifest.json")
```

F: golden-labels archive (original-6 labels live here; C: working copies were removed)

```
F_GOLDEN = r"F:\[Khelsutra] GoPro Hero Black 12\KhelSutra\Training\Golden Labelled"
```

```
BUCKET = "gs://khelsutra-rally-corpus"
```

```
OUT_MANIFEST = os.path.join(REPO, "scratch", "golden.gcs.json")
```

```
OUT_PLAN = os.path.join(REPO, "scratch", "upload_plan.tsv")
```

```
def _resolve_label(spec):
```

```
    """Return (upload_src, date_str). The date = EARLIEST mtime across all known copies (C:
```

```
working
```

```
    copy + F: archive) ? the label's authoring date ? robust to copy-resets (a restored C: copy has
```

```
    today's mtime; the F: archive keeps the real June date). Upload from the C: copy if present."""
```

```
    cands = []
```

```
    p = str(spec["labels"])
```

```
    if os.path.isfile(p):
```

```
        cands.append(p)
```

```
    alt = os.path.join(F_GOLDEN, os.path.basename(p)) # original-6 (+ backed-up new) archive
```

```
copy
```

```
    if os.path.isfile(alt):
```

```
        cands.append(alt)
```

```
    if not cands:
```

```
        return None, ""
```

```
    date = min(datetime.datetime.fromtimestamp(os.path.getmtime(c)) for c in
```

```
cands).strftime("%Y-%m-%d")
```

```
    return cands[0], date
```

```
def main():
```

```
    with open(MANIFEST, encoding="utf-8") as f:
```

```
        specs = json.load(f)
```

```
    plan = [] # (src, dst) upload pairs
```

```
    new_manifest = []
```

```
    problems = []
```

```
    print(f"{'name':<20} {'date':<10} {'traj':<6} {'label':<6} label_src")
```

```
    for s in specs:
```

```

name = s["name"]
traj = str(s["trajectory"])
label, date = _resolve_label(s)
traj_ok = os.path.isfile(traj)
if not traj_ok:
    problems.append(f"MISSING trajectory: {name} -> {traj}")
if label is None:
    problems.append(f"MISSING label: {name} -> {s['labels']}")
print(f"{name:<20} {date:<10} {'ok' if traj_ok else 'MISS':<6} "
      f"{'ok' if label else 'MISS':<6} {label or s['labels']}")

gcs_traj = f"{BUCKET}/trajectories/{name}.csv"
gcs_label = f"{BUCKET}/labels/{date}_{name}.rallies.csv" if label else None
if traj_ok:
    plan.append((traj, gcs_traj))
if label:
    plan.append((label, gcs_label))
new_manifest.append({
    "name": name,
    "trajectory": gcs_traj,
    "fps": s["fps"],
    "frame_width": s["frame_width"],
    "labels": gcs_label,
})

with open(OUT_MANIFEST, "w", encoding="utf-8") as f:
    json.dump(new_manifest, f, indent=2)
with open(OUT_PLAN, "w", encoding="utf-8") as f:
    for src, dst in plan:
        f.write(f"{src}\t{dst}\n")

print(f"\n{len(plan)} objects to upload ({sum(1 for _, d in plan if '/trajectories/' in d)}
traj, "
      f"{sum(1 for _, d in plan if '/labels/' in d)} labels)")
print(f"manifest -> {OUT_MANIFEST} (then gs to {BUCKET}/manifests/golden.json)")
print(f"plan -> {OUT_PLAN}")
if problems:
    print("\n!! PROBLEMS:")
    for p in problems:
        print(" " + p)
else:
    print("\nall trajectories + labels resolved OK")

if __name__ == "__main__":
    main()

```

28. assistant (2026-06-22T10:26:45.373Z)

Now I have the full picture. The `golden.gcs.json` is the actual pushed manifest and it correctly uses the canonical naming with fps/frame_width preserved. The migration script is sound. Let me do final checks: the deletion safety (whether old labels are recoverable), the bracket-glob handling in the staging script, and whether the manifest `name` keys match what eval consumers join on (e.g. served_gate_a uses `name` only for display, joins by spec keys directly ? so name mismatch isn't fatal there, but the trajectory rename `.csv` could mismatch the `--trajectory-source detector` path which joins video stem to label name).

Let me verify the staging script's bracket handling and the train-path stem-join concern.

29. user (2026-06-22T10:26:47.064Z)

```

===stage_corpus_to_gcs.sh===
#!/usr/bin/env bash

```

Stage the Gen-0 rally TRAINING corpus from a local box -> the GCS bucket, so `trainin

runs on a GCP GPU VM instead of local WSL. Idempotent. Run from the repo root on t
The bucket + rally-worker SA are created by provision.sh (default gs://khelsutra-rally-c

Gen-0 training is CPU/numpy (no torch) ? it needs only TIER-1 (trajectories + labels). T
weights + videos) is for the GPU DETECTION pass that PRODUCES trajectories for ne

Override the local sources via env (defaults match the reference dev box):

RALLY_GCS_BUCKET target bucket name (default: khelsutra-rally-corpus)
RALLY_TRAJ_DIR dir of *_traj.csv trajectories (default: the dev-box Annotation S
RALLY_LABELS_GLOB glob of golden *.rallies.csv (default: output/*.rallies.csv)
RALLY_MANIFEST local corpus manifest to remap (default: output/human_lovo_

```
set -uo pipefail
BUCKET="${RALLY_GCS_BUCKET:-khelsutra-rally-corpus}"
TRAJ_DIR="${RALLY_TRAJ_DIR:-C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories}"
LABELS_GLOB="${RALLY_LABELS_GLOB:-output/*.rallies.csv}"
MANIFEST="${RALLY_MANIFEST:-output/human_lovo_manifest.json}"
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
echo "== staging Gen-0 corpus -> gs://$BUCKET =="
```

0) refresh the VM-relative cloud manifest (drops entries whose trajectory/label is miss

```
python "$HERE/make_cloud_manifest.py" --manifest "$MANIFEST" --out
output/cloud_lovo_manifest.json
```

--- TIER 1: everything Gen-0 TRAINING needs (CPU/numpy; NO video, NO GPU) ---

```
gcloud storage cp "$TRAJ_DIR"/*.csv "gs://$BUCKET/trajectories/"
gcloud storage cp $LABELS_GLOB "gs://$BUCKET/labels/"
gcloud storage cp output/cloud_lovo_manifest.json "gs://$BUCKET/manifests/"
gcloud storage cp eval_baselines/heuristic_lovo_n6.json "gs://$BUCKET/baselines/"
```

--- TIER 2: only if RE-RUNNING WASB DETECTION on the VM (to make trajectories for
WASB weights (~6 MB, MIT). From WSL where the file + gcloud live:

wsl bash -lc "gcloud storage cp ~/models/WASB-SBDT/pretrained_weights/wasb_ba
Video proxies (LARGE, ~tens of GB):

gcloud storage cp F:/KhelSutra_work/19june_proxies/*.mp4 gs://\$BUCKET/videos/

```
echo "== upload done. =="
cat <<EOF
```

NEXT ? on the GCP GPU VM (rally-worker SA reads the bucket; code via git clone or 'git archive'
tarball):

```
mkdir -p corpus/trajectories corpus/labels
gcloud storage cp "gs://$BUCKET/trajectories/*" corpus/trajectories/
gcloud storage cp "gs://$BUCKET/labels/*" corpus/labels/
gcloud storage cp "gs://$BUCKET/manifests/cloud_lovo_manifest.json" .
gcloud storage cp "gs://$BUCKET/baselines/heuristic_lovo_n6.json" eval_baselines/
RALLY_GIT_SHA=$(git rev-parse HEAD 2>/dev/null || cat GIT_SHA 2>/dev/null || echo unknown) \
python -m training.gen0.harness --manifest cloud_lovo_manifest.json \
--floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0 --version
0.1.0
gcloud storage cp -r artifacts/rally_tal_gen0 "gs://$BUCKET/artifacts/"
EOF
```

===upload_plan.tsv (what was actually copied)===

C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/adarsh_avi_full_traj.csv

gs://khelsutra-rally-corpus/trajectories/adarsh_avi_singles.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh and Avi.rallies.csv
gs://khelsutra-rally-corpus/labels/2026-06-08_adarsh_avi_singles.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/kushagra_traj.csv gs://khelsutra-rally-corpus/trajectories/kushagra_singles.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/KushagraYashAviFirstVideo.rallies.csv gs://khelsutra-rally-corpus/labels/2026-06-08_kushagra_singles.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/largetestvideo_traj.csv
gs://khelsutra-rally-corpus/trajectories/largetest_doubles.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/LargeTestVideo.rallies.csv
gs://khelsutra-rally-corpus/labels/2026-06-08_largetest_doubles.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/gbaaddy_traj.csv gs://khelsutra-rally-corpus/trajectories/gbaaddy.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/gBaaddy20May2026.rallies.csv
gs://khelsutra-rally-corpus/labels/2026-06-08_gbaaddy.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/testlarge_traj.csv
gs://khelsutra-rally-corpus/trajectories/testlarge_short.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/TestLargeVideo.rallies.csv
gs://khelsutra-rally-corpus/labels/2026-06-08_testlarge_short.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/mahadevpura_traj.csv
gs://khelsutra-rally-corpus/trajectories/mahadevpura_singles.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/MahadevpuraSingles.rallies.csv
gs://khelsutra-rally-corpus/labels/2026-06-09_mahadevpura_singles.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/mahadevpura2_june12_traj.csv
gs://khelsutra-rally-corpus/trajectories/mahadevpura_2.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/mahadevpura_2.rallies.csv
gs://khelsutra-rally-corpus/labels/2026-06-16_mahadevpura_2.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/GX010128_june12_traj.csv
gs://khelsutra-rally-corpus/trajectories/GX010128.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/GX010128.rallies.csv gs://khelsutra-rally-corpus/labels/2026-06-16_GX010128.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/mahadevpura1_june12_traj.csv
gs://khelsutra-rally-corpus/trajectories/mahadevpura_1.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/mahadevpura_1.rallies.csv
gs://khelsutra-rally-corpus/labels/2026-06-17_mahadevpura_1.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/GX030094_june15_traj.csv
gs://khelsutra-rally-corpus/trajectories/GX030094.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/GX030094.rallies.csv gs://khelsutra-rally-corpus/labels/2026-06-17_GX030094.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv
gs://khelsutra-rally-corpus/trajectories/Badminton_BXH_2.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/Badminton_BXH_2.rallies.csv
gs://khelsutra-rally-corpus/labels/2026-06-17_Badminton_BXH_2.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/GX010141_june17_traj.csv
gs://khelsutra-rally-corpus/trajectories/GX010141.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/GX010141.rallies.csv gs://khelsutra-rally-corpus/labels/2026-06-18_GX010141.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/GX010137_june17_traj.csv
gs://khelsutra-rally-corpus/trajectories/GX010137.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/GX010137.rallies.csv gs://khelsutra-rally-corpus/labels/2026-06-18_GX010137.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/Boxhill_Doubles_june18_traj.csv
gs://khelsutra-rally-corpus/trajectories/Boxhill_Doubles.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/Boxhill_Doubles.rallies.csv
gs://khelsutra-rally-corpus/labels/2026-06-18_Boxhill_Doubles.rallies.csv
C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/GX020094_june18_traj.csv
gs://khelsutra-rally-corpus/trajectories/GX020094.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/GX020094.rallies.csv gs://khelsutra-rally-corpus/labels/2026-06-18_GX020094.rallies.csv

30. assistant (2026-06-22T10:26:55.611Z)

Now I have the complete picture for a thorough risk audit. Key observations:

1. ****Two divergent staging paths exist****: `stage\_corpus\_to\_gcs.sh` copies trajectories with their ORIGINAL `\_traj.csv` names and labels via `output/\*.rallies.csv` glob, building `cloud\_lovo\_manifest.json` with `corpus/...` relative paths ? this is INCONSISTENT with the actual `migrate\_golden\_to\_gcs.py` path that renamed to canonical `.csv` / `>\_<name>.rallies.csv` and built `golden.gcs.json`. Two competing migration scripts that produce DIFFERENT bucket layouts.

2. ****The 7 deleted labels**** map to the canonical names via the upload_plan ? they were old-naming dupes. The source labels still exist on F: and `output/\*.rallies.csv` (gitignored but present locally), so recoverability hinges on those non-bucket copies.

3. ****`labels/` rename loses the original filename?name mapping**** unless the manifest survives. The manifest is the ONLY join key now.

Let me confirm the train-path detector join risk (stem matching) and whether `manifests/golden.json` is consumed by `cmd\_train` (it is NOT ? train rebuilds, per doc).

31. user (2026-06-22T10:26:58.276Z)

===make_cloud_manifest.py (the OTHER manifest builder)===

"""Remap a local Gen-0 corpus manifest -> a CLOUD manifest with VM-relative paths.

`training/gen0/harness.py` opens each entry's ``trajectory`` and ``labels`` as LOCAL files (it never reads ``gs://``). A dev-box manifest carries absolute Windows paths (`C:/...`) that don't exist on a Linux GPU VM, so this rewrites every entry to repo-relative paths under ``--corpus-prefix`` (default ``corpus/{trajectories,labels}/<basename>``) ? exactly where `stage\_corpus\_to\_gcs.sh`'s VM-side fetch lands the files. Entries whose trajectory OR label is absent locally are DROPPED (so the cloud run reflects the honest n and can't fail on a missing file).

```
python deploy/gcp/make_cloud_manifest.py \
    --manifest output/human_lovo_manifest.json --out output/cloud_lovo_manifest.json
"""
from __future__ import annotations

import argparse
import json
import os
from typing import Any, Dict, List, Tuple
```

```
def remap(src: List[Dict[str, Any]], corpus_prefix: str
) -> Tuple[List[Dict[str, Any]], List[Tuple[str, str]]]:
    """Return (kept, dropped). ``kept`` entries have VM-relative trajectory/labels paths;
    ``dropped`` is ``[(name, reason)]`` for entries missing a local trajectory or label."""
    kept: List[Dict[str, Any]] = []
    dropped: List[Tuple[str, str]] = []
    for v in src:
        traj_ok = os.path.exists(v["trajectory"])
        lab_ok = os.path.exists(v.get("labels", ""))
        if traj_ok and lab_ok:
            kept.append({
                **{k: v[k] for k in ("name", "fps", "frame_width")},
                "trajectory":
                    f"{corpus_prefix}/trajectories/{os.path.basename(v['trajectory'])}",
                "labels": f"{corpus_prefix}/labels/{os.path.basename(v['labels'])}",
            })
        else:
            dropped.append((v["name"], "no-trajectory" if not traj_ok else "no-label"))
    return kept, dropped
```

```
def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
    ap.add_argument("--out", default="output/cloud_lovo_manifest.json")
    ap.add_argument("--corpus-prefix", default="corpus",
                    help="VM-relative dir the fetch step populates (default: corpus)")
    args = ap.parse_args()

    with open(args.manifest, encoding="utf-8") as f:
        src = json.load(f)

===where the original-6 C: labels went===
Adarsh and Avi.rallies.csv
KushagraYashAviFirstVideo.rallies.csv
LargeTestVideo.rallies.csv
MahadevpuraSingles.rallies.csv
TestLargeVideo.rallies.csv
gBaaddy20May2026.rallies.csv
```

32. assistant (2026-06-22T10:27:04.549Z)

I now have everything needed. The original-6 labels DO still exist on C: (and F:), so the deletion of the 7 bucket label objects is recoverable. The `make\_cloud\_manifest.py` references a non-existent path `deploy/gcp/make\_cloud\_manifest.py` in its docstring (it lives in scratch/). And critically there are TWO incompatible staging scripts producing TWO different manifests (`golden.gcs.json`/`golden.json` canonical-named vs `cloud\_lovo\_manifest.json` basename-preserving with `corpus/` relative paths) ? the train harness reads neither gs:// directly.

Let me do one final verification of the egress/storage cost claim and the `name` field's role in nightly regression consumers to rank severity accurately.

33. user (2026-06-22T10:27:05.891Z)

```
scripts\nightly_regression.py:72:DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output",
"human_lovo_manifest.json")
training\gen0\harness.py:3:    python -m training.gen0.harness --manifest
output/human_lovo_manifest.json \
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py:26:MANIFEST =
os.path.join(MAIN, "output", "human_lovo_manifest.json")
backend\eval\fusion_golden.py:12:    python -m backend.eval.fusion_golden --manifest
output/human_lovo_manifest.json --out-dir output
backend\eval\fusion_golden.py:182:    ap.add_argument("--manifest",
default="output/human_lovo_manifest.json")
backend\eval\rally_seg_eval.py:15:GROUND TRUTH: the golden manifest
(``output/human_lovo_manifest.json``) gives each video's
backend\eval\rally_seg_eval.py:32:DEFAULT_MANIFEST = os.path.join("output",
"human_lovo_manifest.json")
backend\eval\rally_seg_eval.py:46:#: (``output/human_lovo_manifest.json``) is gitignored, so
this committed set is the AUDITABLE source
backend\eval\rally_seq_proto.py:26:    python -m backend.eval.rally_seq_proto --manifest
output/human_lovo_manifest.json
backend\eval\fusion_audio.py:11:    python -m backend.eval.fusion_audio --manifest
output/human_lovo_manifest.json --features-dir output --augment --compare
backend\eval\fusion_audio.py:108:    ap.add_argument("--manifest",
default="output/human_lovo_manifest.json")
backend\eval\served_gate_a.py:20:    python -m backend.eval.served_gate_a --manifest
output/human_lovo_manifest.json
backend\eval\served_gate_a.py:191:    ap.add_argument("--manifest",
default="output/human_lovo_manifest.json")
backend\eval\serve_contrast.py:23:    python -m backend.eval.serve_contrast --manifest
output/human_lovo_manifest.json
backend\eval\serve_contrast.py:145:    ap.add_argument("--manifest",
default="output/human_lovo_manifest.json",
tests\test_served_gate_a_regression.py:59:    os.path.join(_REPO_ROOT, "output",
"human_lovo_manifest.json"),
```

34. assistant (2026-06-22T10:27:11.638Z)

This is the final confirming evidence. ****Every eval/litmus consumer defaults to `output/human\_lovo\_manifest.json`**** (local C:-absolute paths) ? none reads the GCS `manifests/golden.json`. The served Gate-A regression test (``test_served_gate_a_regression.py:59``) is pinned to the local manifest. So Phase-3 (flipping consumers to GCS) is genuinely not done, and the bucket manifest is currently an orphan that no code path consumes ? meaning its format correctness is ****untested in CI****. The ``name`` field is load-bearing for these consumers (rally_seg_eval keys per-video results by it).

I have enough to deliver the ranked risk audit.

35. user (2026-06-22T10:28:01.835Z)

Structured output provided successfully

36. assistant (2026-06-22T10:28:04.036Z)

I've completed the risk/safety audit and returned the structured verdict.